
Universidad Técnica Estatal de Quevedo

Carrera de Ingeniería de Software

GA — Práctica Experimental

Unidad II

Desarrollo de Aplicaciones Web con Tecnologías de Servidor

*PHP 8.x y Java/JSP (Jakarta EE) — Autenticación, CRUD y Controles
OWASP*

Asignatura: Aplicaciones Web

Código / Paralelo: SOFT-R-A | 5to Nivel

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Equipo: D

Integrantes:

CASTRO ESPINOZA KEVIN MOISES

ESCUDERO PLAZA MARIA DEL ROSARIO

LOOR MEDRANDA MARLON TAYLOR

Período: UTEQ 2026 | Corte 1

Fecha: 19 de junio de 2026

Índice

1. Introducción	3
2. Marco Teórico	3
2.1. Tecnologías de Servidor e Integración de Arquitecturas	3
2.1.1. El modelo CGI clásico y sus limitaciones históricas	3
2.1.2. Módulos integrados en proceso: <code>mod_php</code>	4
2.1.3. FastCGI y PHP-FPM: procesos aislados con <i>pools</i> dinámicos . .	4
2.1.4. Servidores de aplicaciones embebidos: Apache Tomcat 10	5
2.1.5. Avances clave de PHP 8.0–8.3	5
2.2. Autenticación Segura y Arquitectura de Sesiones	6
2.2.1. Funciones criptográficas de hashing de contraseñas	6
2.2.2. Ataques de tiempo y comparación en tiempo constante	7
2.2.3. Fijación y secuestro de sesión: controles de mitigación	8
2.2.4. Sesiones con estado (<i>stateful</i>) vs. JWT (<i>stateless</i>)	8
2.2.5. Vulnerabilidades OWASP Top 10 aplicadas al módulo de auten- ticación	9
2.3. Acceso Seguro a Datos, PDO y el Patrón Repository	11
2.3.1. Análisis multi-fase de SQL Injection y puntuación CVSS v3.1 .	11
2.3.2. Mecanismo de dos fases de las sentencias preparadas	12
2.3.3. El patrón Repository y separación de responsabilidades	13
2.4. Controles OWASP y Mitigaciones Técnicas	14
2.4.1. Prevención de XSS (<i>Cross-Site Scripting</i>)	14
2.4.2. Tokens CSRF con verificación en tiempo constante	15
2.4.3. Cabeceras de seguridad HTTP y defensa en profundidad	16
2.5. Arquitectura de Seguridad: Spring Boot y ASP.NET Core	18
2.5.1. Pipeline de middleware: ASP.NET Core vs Spring Security . . .	19
2.5.2. Inyección de Dependencias: ciclos de vida <i>Transient</i> , <i>Scoped</i> y <i>Singleton</i>	20
2.5.3. Spring IoC Container y Spring Security: comparativa con Jakarta EE	21
2.5.4. Principios SOLID aplicados al backend del PFC	23
3. Arquitectura del Sistema	25
3.1. Repositorio Centralizado y Flujo de Trabajo Git	25
3.2. Esquema de Base de Datos MySQL	25
3.3. Estructura de la Aplicación PHP	26
3.4. Estructura de la Aplicación Java/JSP	27
4. Tabla Comparativa: PHP 8.x vs. Java/JSP (Jakarta EE)	30
5. Conclusiones	34
6. Anexo C: Evaluación Analítica	35
6.1. Análisis de Seguridad por Defecto	35
6.2. Evaluación de Factibilidad de Mercado en Ecuador	35
6.3. Síntesis de Análisis de Código Estático con PHPStan	36
6.4. Integración de Arquitectura de Sistemas con PE-U1	38

Referencias	40
Referencias en Formato APA	42

1. Introducción

Repositorio del proyecto: <https://github.com/mloorm14/practica-experimental-2-web.git>

Contiene el código fuente completo de ambas implementaciones (PHP 8.x y Java/JSP), el esquema de base de datos en `database/schema.sql` y los documentos ADR en `docs/adr/`.

El desarrollo de aplicaciones web del lado del servidor constituye uno de los pilares fundamentales de la Ingeniería de Software moderna. Según las estadísticas de W3Techs, PHP es utilizado en más del 77 % de los sitios web cuyo lenguaje de servidor es conocido, mientras que las plataformas Java representan un segmento significativo en aplicaciones empresariales de alta disponibilidad [1].

La presente práctica experimental corresponde a la **Unidad II** de la asignatura *Aplicaciones Web* y tiene como objetivo construir una **aplicación dinámica completa** que integre dos tecnologías de servidor —**PHP 8.x** y **Java/JSP con Jakarta EE**— sobre una base de datos MySQL compartida. Ambas implementaciones deben incluir autenticación segura (registro y *login*), operaciones CRUD con sentencias preparadas y los controles de seguridad obligatorios establecidos por el estándar **OWASP Top 10** [2].

Un principio transversal, válido independientemente del lenguaje de servidor —ya sea PHP, Java, PERL o cualquier otro—, es que el ciclo *petición-respuesta* de HTTP es **sin estado** (*stateless*): el servidor no recuerda al usuario entre una petición y la siguiente. Por ello, mecanismos como las sesiones (`$_SESSION` en PHP, `HttpSession` en Jakarta EE) son imprescindibles para mantener la identidad del usuario autenticado entre cada *request*.

Los entregables de esta práctica son: el código fuente de ambas aplicaciones, la base de datos MySQL con su esquema, y el presente informe técnico con portada, desarrollo, tabla comparativa, conclusiones y referencias en formato IEEE.

2. Marco Teórico

2.1. Tecnologías de Servidor e Integración de Arquitecturas

La relación entre el servidor web y el lenguaje de *backend* ha evolucionado a través de cuatro modelos arquitectónicos fundamentales, cada uno con implicaciones directas sobre el rendimiento, el aislamiento de procesos y la seguridad. Comprender esta evolución es esencial para justificar las decisiones de diseño tomadas en esta práctica.

2.1.1. El modelo CGI clásico y sus limitaciones históricas

El **Common Gateway Interface** (CGI) fue la primera especificación estandarizada para que los servidores web ejecutasen programas externos y devolviesen su salida como respuesta HTTP [3]. El mecanismo es conceptualmente simple: ante cada petición HTTP, el servidor crea un **proceso hijo nuevo**, le transfiere el contexto de

la petición mediante **variables de entorno** (como `REQUEST_METHOD`, `QUERY_STRING`, `CONTENT_TYPE` y `HTTP_COOKIE`), y le conecta la entrada estándar (`stdin`) para cuerpos POST y la salida estándar (`stdout`) para la respuesta. El proceso CGI escribe primero las cabeceras HTTP separadas por `\n\n` y luego el cuerpo, tras lo cual termina y libera todos sus recursos.

PERL fue el lenguaje que dominó el ecosistema CGI en los años noventa gracias al módulo `CGI.pm`, el cual abstraía la lectura de parámetros (`param('campo')`), la generación de cabeceras (`header()`) y el escape HTML (`escapeHTML()`). Sin embargo, el modelo presentaba **tres limitaciones críticas**: (1) el *overhead* de creación de un proceso UNIX por cada petición, que bajo carga concurrente producía tiempos de respuesta inaceptables; (2) la **ausencia de estado persistente** entre peticiones: no existía *pool* de conexiones de base de datos, por lo que cada *script* debía establecer y cerrar su propia conexión; (3) la carga reiterada del intérprete PERL junto con todos los módulos importados (`DBI`, `CGI.pm`, `Template`) en cada invocación.

A pesar de estas limitaciones, CGI conserva nichos modernos legítimos: sistemas embebidos con servidores HTTP minimalistas (como `lighttpd` sobre enrutadores de red), aplicaciones de bajo tráfico donde la simplicidad supera al rendimiento, y entornos de escritura de *scripts* de administración de larga data que no justifican migración.

2.1.2. Módulos integrados en proceso: `mod_php`

La primera respuesta a las limitaciones de CGI fue integrar el intérprete directamente dentro del proceso del servidor web. `mod_php` es un módulo de Apache que ejecuta el intérprete PHP en el **mismo espacio de memoria** que el proceso de Apache (modelo `prefork`) o dentro de uno de sus hilos (modelo `worker/event`). El resultado es que la carga del intérprete ocurre una sola vez al iniciar Apache, y cada petición simplemente invoca la función de ejecución del módulo ya residente en memoria.

El *overhead* por petición se reduce drásticamente: no hay `fork()`, no hay carga de librerías, y el motor Zend (motor de ejecución de PHP) ya está inicializado. La desventaja es el **acoplamiento**: si un proceso Apache se corrompe por un *bug* en un *script* PHP, puede afectar a otras peticiones servidas por ese mismo proceso. Además, en el modelo `prefork`, cada proceso Apache consume la memoria completa del intérprete PHP aunque esté sirviendo un recurso estático.

2.1.3. FastCGI y PHP-FPM: procesos aislados con *pools* dinámicos

FastCGI es un protocolo que resuelve el acoplamiento de `mod_php` manteniendo la eficiencia. En lugar de un proceso por petición (CGI) o un módulo en proceso (`mod_php`), FastCGI establece un **conjunto de procesos persistentes** del intérprete que se comunican con el servidor web a través de un *socket* Unix o TCP. El servidor web (típicamente Nginx, aunque también Apache con `mod_fastcgi`) actúa exclusivamente como proxy: recibe la petición HTTP, la reenvía al proceso FastCGI disponible y retransmite la respuesta.

PHP-FPM (*PHP FastCGI Process Manager*) es la implementación oficial de FastCGI para PHP e incluye un gestor de *pools* con configuración granular [4]: número mínimo y máximo de procesos hijo, estrategia de escalado (`pm = dynamic | static |`

ondemand), límites de memoria por proceso y rotación automática de procesos tras un número configurable de peticiones atendidas (para mitigar fugas de memoria). Este modelo es el estándar de facto en arquitecturas modernas de alta disponibilidad con PHP.

2.1.4. Servidores de aplicaciones embebidos: Apache Tomcat 10

El cuarto modelo, utilizado por Java/JSP en esta práctica, es el **servidor de aplicaciones embebido**. **Apache Tomcat 10** implementa las especificaciones Jakarta Servlet 6.0 y Jakarta Pages 3.1 [5] y actúa como contenedor de *servlets*: ejecuta el *bytecode* compilado (*.class* / *.war*) dentro de la **JVM**, gestionando un *pool* de hilos para atender peticiones concurrentes. A diferencia de PHP, la JVM es un proceso persistente de larga vida: mantiene el estado de la aplicación entre peticiones (conexiones de base de datos en *pool*, caché, configuración) y optimiza el *bytecode* en tiempo de ejecución mediante el compilador **JIT** (*Just-In-Time*) de la JVM.

El ciclo de vida de un Servlet es preciso: `init()` se llama una vez al desplegar el *.war*; `service()` (que delega en `doGet()` o `doPost()`) se invoca por cada petición entrante en un hilo del *pool*; `destroy()` se ejecuta al desmontar la aplicación. Este modelo **multi-hilo y con estado** ofrece mejor rendimiento sostenido que PHP bajo cargas altas, al precio de mayor complejidad operativa (gestión de la JVM, *classpath*, despliegue de *.war*, configuración de *context.xml*) [6].

2.1.5. Avances clave de PHP 8.0–8.3

La versión PHP 8.0 introdujo el **compilador JIT** (*Just-In-Time*), integrado en el motor Zend, que compila *opcodes* a código máquina nativo en tiempo de ejecución, mejorando el rendimiento en tareas CPU-intensivas [7]. Las **expresiones match** reemplazan a **switch** con comparación estricta por identidad (`===`), sin *fall-through* implícito y con valor de retorno directo. Los **tipos unión** (`int|string`) y los tipos de intersección (`Countable&Iterator`, PHP 8.1) habilitan anotaciones de tipo más precisas. PHP 8.1 además introdujo las **propiedades readonly**, que sólo pueden asignarse una vez en el constructor, modelando inmutabilidad de valor sin librerías externas. PHP 8.2 extendió el concepto a **clases readonly** completas. Finalmente, las **Fibras** (`Fiber`, PHP 8.1) proporcionan un mecanismo de concurrencia cooperativa: permiten pausar y reanudar la ejecución de una rutina sin bloquear el hilo del proceso, abriendo la puerta a marcos de trabajo asíncronos como *ReactPHP* o *Amp* sin requerir extensiones externas de corrutinas. Estas mejoras posicionan a PHP 8.x como un lenguaje con garantías de tipado comparables a las de Java en el contexto de proyectos que adoptan herramientas de análisis estático como PHPStan [8].

Resumen comparativo de modelos de integración: CGI crea un proceso por petición (alto overhead, máximo aislamiento); `mod_php` integra el intérprete en Apache (mínimo overhead, acoplamiento de proceso); PHP-FPM mantiene pools de procesos aislados comunicados por FastCGI (equilibrio óptimo); Tomcat/JVM ejecuta bytecode en un pool de hilos persistente (máximo rendimiento sostenido, mayor complejidad operativa).

2.2. Autenticación Segura y Arquitectura de Sesiones

La autenticación es la primera línea de defensa de cualquier aplicación web. Un diseño deficiente en esta capa compromete todas las capas subsiguientes, independientemente de la robustez del resto de controles.

2.2.1. Funciones criptográficas de hashing de contraseñas

Nunca deben almacenarse contraseñas en texto plano ni con funciones hash de propósito general como MD5 o SHA-1 [9]. Estas funciones fueron diseñadas para ser rápidas (miles de millones de hashes por segundo en GPU modernas), lo que las hace inadecuadas para almacenamiento de contraseñas: un atacante que obtenga la base de datos puede ejecutar ataques de diccionario o fuerza bruta a velocidades inaceptables.

Las funciones especializadas para hashing de contraseñas incorporan deliberadamente un **coste computacional configurable**:

- **bcrypt** (Provos & Mazières, 1999): basado en el cifrado de bloque Blowfish, opera con un parámetro de **factor de trabajo** (*work factor* o *cost*), habitualmente entre 10 y 14, que define el número de rondas de expansión de clave como 2^{cost} . Con *cost*=12, bcrypt realiza 4,096 rondas, tardando aproximadamente 250 ms en hardware moderno de servidor, lo que limita los intentos de fuerza bruta a pocos miles por segundo. Su principal limitación es que acepta como máximo 72 bytes de contraseña efectiva, truncando silenciosamente cualquier exceso.
- **Argon2** (ganador del Password Hashing Competition, 2015): algoritmo moderno con tres variantes [10]. **Argon2i** está optimizado para resistencia a ataques de canal lateral (análisis de acceso a memoria) y es preferido en contextos de derivación de claves criptográficas. **Argon2d** maximiza la resistencia a GPU mediante accesos a memoria dependientes del dato. **Argon2id** (la variante recomendada por defecto) combina ambas estrategias en dos pasadas, ofreciendo protección tanto contra ataques de canal lateral como contra hardware especializado. Sus parámetros configurables son:
 - **memory_cost**: kilobytes de memoria RAM que debe consumir el algoritmo (mínimo recomendado: 65 536 KB = 64 MB para 2024).
 - **time_cost**: número de iteraciones o pasadas sobre la memoria.
 - **parallelism** (lanes): número de hilos paralelos usados internamente.

Esta combinación de **coste de tiempo y coste de memoria** hace que Argon2id sea resistente a ataques basados en ASICs (circuitos integrados de aplicación específica) y FPGAs, ya que la gran cantidad de memoria requerida encarecería prohibitivamente el hardware especializado.

Listing 1: PHP: Comparativa de algoritmos de hashing – bcrypt vs Argon2id

```
1 <?php
2 declare(strict_types=1);
3
4 $password = 'ContrasenaDelUsuario2024!';
5
6 // bcrypt (PASSWORD_BCRYPT): cost=12, trunca a 72 bytes
7 $hash_bcrypt = password_hash($password, PASSWORD_BCRYPT, ['cost' => 12]);
8 // Resultado: $2y$12$<22-char-salt><31-char-hash> -- 60 caracteres totales
9
```

```

10 // Argon2id (recomendado desde PHP 7.3+):
11 $hash_argon2id = password_hash($password, PASSWORD_ARGON2ID, [
12     'memory_cost' => 65536,    // 64 MB de RAM por hash
13     'time_cost'   => 4,        // 4 iteraciones
14     'threads'     => 2,        // 2 hilos paralelos (lanes)
15 ]);
16 // Resultado: $argon2id$v=19$m=65536,t=4,p=2$<salt>$<hash>
17
18 // Verificacion (independiente del algoritmo usado al crear el hash)
19 if (password_verify($password, $hash_argon2id)) {
20     echo 'Autenticacion exitosa';
21 }
22
23 // Rehash automatico si los parametros cambian en el futuro
24 if (password_needs_rehash($hash_argon2id, PASSWORD_ARGON2ID,
25     ['memory_cost' => 131072, 'time_cost' => 4, 'threads' => 2])) {
26     $nuevo_hash = password_hash($password, PASSWORD_ARGON2ID, [...]);
27     // Actualizar en base de datos
28 }

```

2.2.2. Ataques de tiempo y comparación en tiempo constante

Un **ataque de tiempo** (*timing attack*) es un ataque de canal lateral que explota el hecho de que las comparaciones de cadenas convencionales en la mayoría de los lenguajes retornan **false** en el primer byte diferente, sin comparar el resto. Un atacante que pueda medir con precisión el tiempo de respuesta del servidor puede inferir cuántos bytes iniciales de su intento coinciden con el valor correcto, reduciendo el espacio de búsqueda de fuerza bruta de manera significativa.

La solución es utilizar **comparación en tiempo constante**: una función que siempre recorre la totalidad de ambas cadenas, independientemente de cuántos bytes coincidan, devolviendo el resultado sólo al final. En PHP, `password_verify()` ya implementa internamente esta comparación constante [11]. Para comparar tokens CSRF u otros valores secretos arbitrarios, se usa `hash_equals($conocido, $recibido)`, que garantiza tiempo de ejecución $O(n)$ siempre. En Java, `BCrypt.checkpw()` de `jBCrypt` y el método `MessageDigest.isEqual(byte[], byte[])` (disponible desde Java 6) proporcionan la misma garantía de tiempo constante, mitigando estos ataques a nivel de implementación de la librería.

Listing 2: Java: BCrypt y comparación en tiempo constante

```

1 import org.mindrot.jbcrypt.BCrypt;
2 import java.security.MessageDigest;
3
4 // Registro: generar salt con factor de trabajo 12
5 String hash = BCrypt.hashpw(passwordIngresada, BCrypt.gensalt(12));
6 // Almacenar 'hash' en la base de datos (columna VARCHAR(60))
7
8 // Login: checkpw() usa comparacion en tiempo constante internamente
9 boolean valido = BCrypt.checkpw(passwordIngresada, hashAlmacenado);
10
11 // Para tokens CSRF: MessageDigest.isEqual() tiempo constante
12 byte[] tokenSesion = sessionToken.getBytes("UTF-8");
13 byte[] tokenFormular = formularioToken.getBytes("UTF-8");
14 boolean igualTokens = MessageDigest.isEqual(tokenSesion, tokenFormular);

```


2.2.3. Fijación y secuestro de sesión: controles de mitigación

La **fijación de sesión** (*Session Fixation*) ocurre cuando un atacante fuerza al cliente a utilizar un identificador de sesión conocido por él antes del proceso de autenticación, de modo que tras el *login* exitoso el servidor asocia las credenciales al identificador ya conocido por el atacante. La mitigación es obligatoria: **regenerar el ID de sesión inmediatamente después de la autenticación exitosa**, lo que invalida el identificador previo.

El **secuestro de sesión** (*Session Hijacking*) ocurre cuando un atacante obtiene el valor de la *cookie* de sesión (PHPSESSID o JSESSIONID) de una víctima autenticada. Los vectores de obtención incluyen XSS, sniffing de red en HTTP no cifrado, y acceso físico al navegador. Los controles de configuración de la *cookie* mitigan este riesgo:

- **HttpOnly**: impide que JavaScript en el cliente acceda al valor de la *cookie* (`document.cookie`), neutralizando el vector XSS.
- **Secure**: instruye al navegador a enviar la *cookie* solo en conexiones HTTPS cifradas, eliminando el vector de sniffing.
- **SameSite=Strict**: impide que la *cookie* se incluya en peticiones cross-site, mitiga ataques CSRF en navegadores modernos [12].

Listing 3: PHP: Configuración segura de *cookie* de sesión

```
1 <?php
2 declare(strict_types=1);
3
4 // Configurar antes de session_start()
5 session_set_cookie_params([
6     'lifetime' => 0,           // Cookie de sesion (expira al cerrar navegador)
7     'path'     => '/',
8     'domain'   => '',         // Dominio actual
9     'secure'   => true,       // Solo HTTPS (produccion)
10    'httponly' => true,       // Inaccesible desde JavaScript
11    'samesite'  => 'Strict',   // Bloquea envio cross-site
12]);
13 session_start();
14
15 // Tras autenticacion exitosa: regenerar ID para prevenir fixation
16 session_regenerate_id(true); // true = elimina la sesion anterior del servidor
17 $_SESSION['user_id']      = (int)$fila['id'];
18 $_SESSION['user_nombre']  = $fila['nombre'];
19 $_SESSION['login_time']   = time();
```

2.2.4. Sesiones con estado (*stateful*) vs. JWT (*stateless*)

Las sesiones server-side (PHPSESSID / JSESSIONID) son **con estado**: el servidor almacena los datos de sesión (en disco, Redis u otro backend) y el cliente solo guarda el identificador opaco. La revocación es trivial: eliminar el registro de sesión en el servidor. La desventaja es la **necesidad de almacenamiento compartido** en arquitecturas multi-servidor (clústers), lo que requiere un backend centralizado como Redis.

Los **JSON Web Tokens** (JWT) son **sin estado**: toda la información del usuario se codifica en el *token* y se firma criptográficamente; el servidor no almacena nada. Esto facilita la escalabilidad horizontal, pero introduce una **complejidad crítica en la**

revocación: una vez emitido, un JWT válido no puede invalidarse sin implementar una lista negra (*blacklist*) en algún almacén compartido, lo que anula parcialmente la ventaja *stateless*. Para esta práctica de nivel universitario, las sesiones server-side son la elección arquitectónicamente correcta: más simples, con revocación inmediata y menor superficie de ataque derivada de una mala gestión del *token* en el cliente.

2.2.5. Vulnerabilidades OWASP Top 10 aplicadas al módulo de autenticación

La guía experimental exige documentar explícitamente cómo las vulnerabilidades A01, A02 y A07 del **OWASP Top 10:2021** [2] se manifiestan en el PFC y cómo fueron mitigadas.

A01:2021 — Control de Acceso Roto (*Broken Access Control*)

Qué significa: fallo en la aplicación de restricciones sobre lo que los usuarios autenticados pueden hacer o ver. Incluye acceso a recursos de otros usuarios (*Insecure Direct Object Reference*, IDOR), omisión de verificación de permisos en rutas protegidas, y elevación de privilegios.

Cómo puede manifestarse en el PFC: un usuario autenticado con `user_id = 1` podría intentar acceder a los productos del usuario 2 enviando `GET /product/edit?id=5` si el producto `id=5` pertenece a otro usuario. Sin control, la aplicación devolvería datos que no le corresponden.

Mitigación implementada: todos los accesos a la tabla `productos` incluyen la condición `AND usuario_id = ?` con el `user_id` extraído exclusivamente de la sesión del servidor (nunca del parámetro HTTP), usando sentencias preparadas:

Listing 4: A01 mitigado: control de acceso con `AND usuario_id = ?` en toda consulta

```

1 // JdbcProductoRepository.java -- control de acceso en cada operacion CRUD
2 public Map<String,Object> getByIdAndUsuarioId(int id, int usuarioId) {
3     PreparedStatement ps = conn.prepareStatement(
4         // El AND usuario_id = ? impide que un usuario acceda a productos ajenos
5         "SELECT id, nombre, descripcion, precio, stock " +
6         "FROM productos WHERE id = ? AND usuario_id = ?"
7     );
8     ps.setInt(1, id);
9     ps.setInt(2, usuarioId); // Obtenido de session, nunca del request del usuario
10    // Si el id no pertenece a este usuarioId, el ResultSet estara vacio
11 }
12
13 // EditServlet.java -- userId viene de la sesion del servidor, no del formulario
14 int userId = (Integer) session.getAttribute("user_id");
15 Map<String,Object> producto = repo.getByIdAndUsuarioId(id, userId);
16 if (producto == null) {
17     res.sendRedirect(req.getContextPath() + "/dashboard"); // Redirige sin revelar
18     // info
19     return;
20 }

```

A02:2021 — Fallas Criptográficas (*Cryptographic Failures*)

Qué significa: uso de algoritmos criptográficos débiles o ausencia de cifrado en datos sensibles. Incluye almacenar contraseñas en texto plano o con MD5/SHA-1, transmitir datos sensibles sin HTTPS, y usar generadores de números aleatorios no criptográficamente seguros para tokens.

Cómo puede manifestarse en el PFC: si las contraseñas se almacenaran con MD5 (`md5($password)`), un atacante que obtuviera la tabla `usuarios` podría recuperarlas en segundos con tablas arcoíris. Si los tokens CSRF se generaran con `rand()` en lugar de `random_bytes()`, serían predecibles por un atacante que conozca la semilla del generador.

Mitigación implementada: (1) **Hashing de contraseñas** con `password_hash($password, PASSWORD_BCRYPT, ['cost' => 12])` en PHP y `BCrypt.hashpw(password, BCrypt.gensalt(12))` en Java; (2) **tokens CSRF** generados con `bin2hex(random_bytes(32))` (256 bits de entropía criptográfica) en PHP y `SecureRandom` certificado por FIPS 140-2 en Java; (3) **cabeceras HSTS** preparadas para producción HTTPS en `sendSecurityHeaders()`:

Listing 5: A02 mitigado: hashing seguro y generación criptográfica de tokens

```

1 <?php
2 // includes/auth.php -- A02: NUNCA md5/sha1, siempre bcrypt con cost >= 12
3 $hash = password_hash($password, PASSWORD_BCRYPT, ['cost' => 12]);
4 // El hash resultante: $2y$12$<22-char-salt><31-char-hash>
5 // Imposible revertir a texto plano incluso con GPUs modernas a cost=12
6
7 // includes/csrf.php -- A02: generador criptograficamente seguro
8 $_SESSION['csrf_token'] = bin2hex(random_bytes(32)); // CSPRNG del sistema operativo
9 // random_bytes() usa /dev/urandom en Linux: 256 bits de entropia real
10 // INCORRECTO: rand() o mt_rand() son predecibles y NO son CSPRNG
11
12 // includes/headers.php -- A02: HSTS listo para produccion HTTPS
13 // header('Strict-Transport-Security: max-age=31536000; includeSubDomains;
    preload');
```

A07:2021 — Fallas de Identificación y Autenticación (*Identification and Authentication Failures*)

Qué significa: debilidades en el mecanismo de autenticación que permiten comprometer contraseñas, tokens de sesión o identidades de usuario. Incluye ataques de fuerza bruta sin limitación de intentos, fijación de sesión, mensajes de error que revelan qué campo falló (enumeración de usuarios), y mantenimiento del mismo ID de sesión antes y después del login.

Cómo puede manifestarse en el PFC: un mensaje de error que diga “Contraseña incorrecta” permite a un atacante confirmar que el email existe. Si el ID de sesión no se regenera tras el login, un atacante que haya fijado previamente el ID puede secuestrar la sesión del usuario.

Mitigación implementada: (1) **Mensajes genéricos:** el error siempre dice “Correo electrónico o contraseña incorrectos” sin especificar cuál falló; (2) **Regenera-**

ción de ID: `session_regenerate_id(true)` en PHP y `session.invalidate()` + nueva sesión en Java, inmediatamente tras la autenticación exitosa; (3) **Configuración segura de cookie** con `HttpOnly`, `SameSite=Strict` y `Secure` para producción:

Listing 6: A07 mitigado: mensaje genérico + regeneración de sesión + cookie segura

```

1 <?php
2 // includes/auth.php -- A07: mensaje generico, sin revelar que campo fallo
3 if (!$user || !password_verify($password, $user['password_hash'])) {
4     return false; // El llamador mostrara siempre el mismo mensaje generico
5     // INCORRECTO: "Email no encontrado" o "Contraseña incorrecta" revela info
6 }
7
8 // login.php -- A07: regenerar ID de sesion tras autenticacion exitosa
9 session_regenerate_id(true); // true = destruye la sesion antigua en el servidor
10 $_SESSION['user_id'] = (int)$fila['id'];
11 // Si un atacante habia fijado el ID anterior, ahora ese ID es invalido
12
13 // login.php -- A07: cookie de sesion con flags de seguridad
14 session_set_cookie_params([
15     'httponly' => true, // JavaScript no puede leer PHPSESSID (anti-XSS)
16     'samesite' => 'Strict', // No se envia en peticiones cross-site (anti-CSRF)
17     'secure' => true, // Solo en HTTPS (produccion)
18 ]);

```

2.3. Acceso Seguro a Datos, PDO y el Patrón Repository

2.3.1. Análisis multi-fase de SQL Injection y puntuación CVSS v3.1

La **inyección SQL** (*SQL Injection*, SQLi) ocurre cuando datos suministrados por el usuario se concatenan directamente en una cadena SQL y son interpretados por el motor de base de datos como código en lugar de como datos. Consideremos el ejemplo clásico de un formulario de autenticación:

Listing 7: PHP VULNERABLE: Concatenación directa – NO usar

```

1 <?php
2 // CODIGO VULNERABLE -- EJEMPLO EDUCATIVO -- NUNCA USAR EN PRODUCCION
3 $email = $_POST['email'];
4 $pass = $_POST['password'];
5 $sql = "SELECT * FROM usuarios WHERE email = '$email' AND password = '$pass'";
6 // Si el usuario ingresa: email = admin@test.com' OR '1'='1' --
7 // La cadena SQL resultante es:
8 // SELECT * FROM usuarios WHERE email = 'admin@test.com' OR '1'='1' --' AND
9 // password = ''
10 // La condicion OR '1'='1' siempre es verdadera; el comentario '--' elimina el
11 // resto.
12 // El atacante obtiene acceso como el primer usuario de la tabla.

```

Esta vulnerabilidad tiene una puntuación **CVSS v3.1** que la clasifica como **Crítica** [13]. Calculando el vector base:

- **AV:N** (*Attack Vector: Network*): explorable remotamente.
- **AC:L** (*Attack Complexity: Low*): no requiere condiciones especiales.

- **PR:N** (*Privileges Required: None*): no se requiere autenticación previa.
- **UI:N** (*User Interaction: None*): el ataque es autónomo.
- **S:C** (*Scope: Changed*): puede afectar recursos más allá de la aplicación.
- **C:H / I:H / A:H**: impacto Alto en Confidencialidad, Integridad y Disponibilidad.

Vector CVSS: CVSS: 3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H — Puntuación Base: **10.0 (Crítica)**. Esto confirma que SQLi en un formulario de autenticación representa el máximo riesgo posible según el estándar internacional de cuantificación de vulnerabilidades.

2.3.2. Mecanismo de dos fases de las sentencias preparadas

Las **sentencias preparadas** (*Prepared Statements*) mitigan SQLi mediante una separación temporal entre la definición de la consulta y la vinculación de los datos [14]:

1. **Fase 1 — Compilación**: la cadena SQL con marcadores de posición (?) en PDO/JDBC, o marcadores con nombre como :email en PDO) se envía al motor de base de datos. MySQL analiza la sintaxis, valida los permisos y genera un **plan de ejecución** (*query plan*) que queda almacenado en el servidor. En este momento no hay ningún dato de usuario involucrado.
2. **Fase 2 — Vinculación y ejecución**: los valores reales del usuario se envían al servidor como **parámetros tipados**, completamente separados del texto SQL. El motor los trata siempre como *datos literales* dentro del plan ya compilado, nunca como extensiones de la sintaxis SQL. Si un atacante inyecta ' OR '1'='1, ese valor se pasará como una cadena literal comparada con el campo email, nunca interpretada como código.

En PHP PDO existen dos métodos de vinculación con diferencias semánticas críticas:

Listing 8: PHP/PDO: bindParam() por referencia vs bindValue() por valor

```

1 <?php
2 declare(strict_types=1);
3
4 $pdo = new PDO('mysql:host=localhost;dbname=appweb_unidad2;charset=utf8mb4',
5               DB_USER, DB_PASS,
6               [PDO::ATTR_EMULATE_PREPARES => false, // Sentencias preparadas
7               PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
8               PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]);
9
10 $stmt = $pdo->prepare(
11     'SELECT id, nombre FROM usuarios WHERE email = :email AND activo = :activo');
12
13 // bindParam(): vincula POR REFERENCIA -- la variable se lee al momento de execute()
14 $email = 'usuario@ejemplo.com';
15 $activo = 1;
16 $stmt->bindParam(':email', $email, PDO::PARAM_STR); // Lee $email en execute()
17 $stmt->bindParam(':activo', $activo, PDO::PARAM_INT);
18
19 $email = 'otro@ejemplo.com'; // <-- Si cambiamos $email antes de execute()...
20 $stmt->execute();             // <-- ...se usa 'otro@ejemplo.com', no el original
21
22 // bindValue(): vincula POR VALOR -- copia el valor en el momento de la llamada

```

```

23 $stmt2 = $pdo->prepare(
24     'SELECT id FROM productos WHERE usuario_id = :uid AND precio > :min');
25 $stmt2->bindValue(':uid', $_SESSION['user_id'], PDO::PARAM_INT); // Copia ahora
26 $stmt2->bindValue(':min', 0.0, PDO::PARAM_STR);
27 $stmt2->execute();

```

La elección entre `bindParam()` y `bindValue()` debe hacerse con consciencia: `bindParam()` es útil en bucles donde la variable se actualiza entre iteraciones y se reutiliza el mismo `prepare()`; `bindValue()` es más seguro y predecible para el caso general, evitando errores difíciles de depurar por mutación de variables [15].

2.3.3. El patrón Repository y separación de responsabilidades

El **patrón Repository** es un patrón de acceso a datos que introduce una **capa de abstracción** entre la lógica de negocio y el mecanismo de persistencia. Define una **interfaz** (contrato abstracto) que la lógica de negocio consume, y una **implementación concreta** (*data mapper*) que traduce las operaciones de la interfaz a SQL mediante PDO o JDBC. Este patrón implementa el principio de **Separación de Responsabilidades** (SoC, *Separation of Concerns*) y el principio de **Inversión de Dependencias** (DIP) de SOLID [16]:

Listing 9: PHP: Patrón Repository con interfaz PDO

```

1 <?php
2 declare(strict_types=1);
3
4 // 1. INTERFAZ: contrato abstracto (la logica de negocio depende solo de esto)
5 interface ProductoRepositoryInterface {
6     /** @return array<int, array<string, mixed>> */
7     public function getAllByUsuarioId(int $usuarioId): array;
8     public function getIdAndUsuarioId(int $id, int $usuarioId): array|false;
9     public function create(string $nombre, string $descripcion,
10         float $precio, int $stock, int $usuarioId): bool;
11     public function update(int $id, string $nombre, string $descripcion,
12         float $precio, int $stock, int $usuarioId): bool;
13     public function delete(int $id, int $usuarioId): bool;
14 }
15
16 // 2. IMPLEMENTACION CONCRETA: traduce la interfaz a SQL con PDO
17 final class PdoProductoRepository implements ProductoRepositoryInterface {
18     public function __construct(private readonly PDO $pdo) {}
19
20     public function getAllByUsuarioId(int $usuarioId): array {
21         $stmt = $this->pdo->prepare(
22             'SELECT id, nombre, descripcion, precio, stock
23             FROM productos WHERE usuario_id = ? ORDER BY id DESC'
24         );
25         $stmt->execute([$usuarioId]);
26         return $stmt->fetchAll(); // PDO::FETCH_ASSOC por defecto en config
27     }
28
29     public function delete(int $id, int $usuarioId): bool {
30         $stmt = $this->pdo->prepare(
31             'DELETE FROM productos WHERE id = ? AND usuario_id = ?'
32         );
33         $stmt->execute([$id, $usuarioId]);

```

```

34     return $stmt->rowCount() === 1; // true solo si se elimino exactamente 1
        fila
35     }
36     // ... resto de metodos omitidos por brevedad en este fragmento
37 }
38
39 // 3. USO EN EL CONTROLADOR: depende de la interfaz, no de la implementacion
40 function handleDelete(PDO $pdo, int $id, int $usuarioId): void {
41     $repo = new PdoProductoRepository($pdo); // Inyeccion de dependencia
42     if (!$repo->delete($id, $usuarioId)) {
43         throw new RuntimeException('Producto no encontrado o sin permiso.');

```

La comparación con el patrón **Active Record** (popularizado por Eloquent en Laravel) es instructiva: en Active Record, cada clase del modelo hereda de una clase base que incluye tanto los datos del objeto como los métodos de persistencia (`User::find()`, `$user->save()`, `$user->delete()`). Esto simplifica el código inicial pero acopla la lógica de negocio a la base de datos: cambiar el ORM o testear unitariamente sin base de datos real exige herramientas de *mocking* complejas. El patrón Repository permite sustituir `PdoProductoRepository` por una implementación en memoria (`InMemoryProductoRepository`) para pruebas unitarias sin tocar la base de datos, aplicando el principio de inversión de dependencias de manera explícita [17].

2.4. Controles OWASP y Mitigaciones Técnicas

2.4.1. Prevención de XSS (*Cross-Site Scripting*)

XSS ocurre cuando datos del usuario se insertan en el HTML de respuesta sin escapar, permitiendo ejecutar JavaScript arbitrario en el navegador de la víctima [18]. La misma regla de oro de la guía de la asignatura (“*escapar toda salida*”) se aplica directamente:

Regla universal: convertir `<`, `>`, `"`, `'`, `&` en sus entidades HTML antes de cualquier salida al navegador. En PHP: `htmlspecialchars($val, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')`. En JSP/JSTL: `<c:out value="{variable}"/>` o `{fn:escapeXml(variable)}`. El principio es idéntico al `CGI::escapeHTML()` y al `escape_html()` estudiados en la guía de PERL de la asignatura.

Listing 10: PHP: Función auxiliar universal de saneamiento anti-XSS

```

1 <?php
2 function esc(mixed $val): string {
3     return htmlspecialchars(
4         (string)$val,
5         ENT_QUOTES | ENT_SUBSTITUTE,
6         'UTF-8'
7     );
8 }
9 echo '<td>' . esc($producto['nombre']) . '</td>';
10 echo '<input value="' . esc($producto['precio']) . '">';

```

Listing 11: JSP/JSTL: Escape automático con `<c:out>`

```

1 <%@ taglib prefix="c" uri="jakarta.tags.core" %>
2 <%@ taglib prefix="fn" uri="jakarta.tags.functions" %>
3 <td><c:out value="\${producto.nombre}"/></td>
4 <input type="text" value="\${fn:escapeXml(producto.precio)}">

```

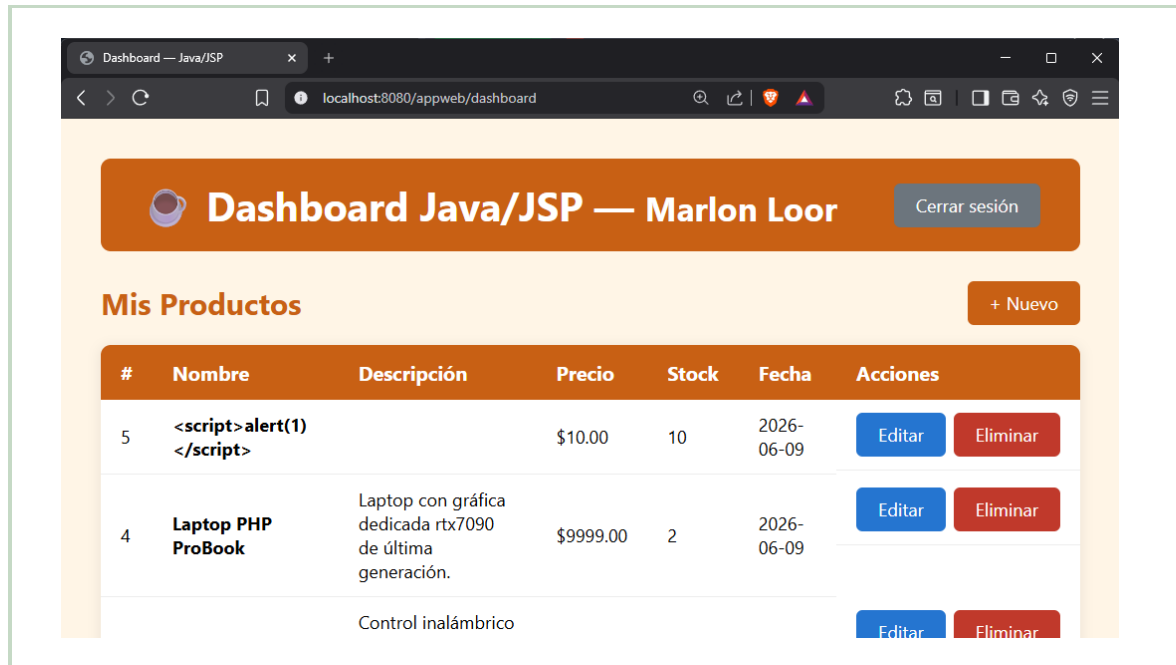


Figura 1: Prueba de XSS: al ingresar `<script>alert(1)</script>` como nombre de producto, la salida HTML muestra la cadena como texto inofensivo escapado. El código JavaScript no se ejecuta en ninguno de los dos ecosistemas.

2.4.2. Tokens CSRF con verificación en tiempo constante

Un ataque CSRF (*Cross-Site Request Forgery*) induce al navegador autenticado de una víctima a enviar peticiones no autorizadas. La defensa estándar combina un **token de alta entropía vinculado a la sesión con validación en tiempo constante** para evitar que el mecanismo de defensa sea a su vez vulnerable a ataques de tiempo [12].

En PHP, `bin2hex(random_bytes(32))` genera 32 bytes criptográficamente aleatorios (256 bits de entropía) produciendo 64 caracteres hexadecimales. La función `hash_equals($conocido, $recibido)` garantiza que la comparación recorre siempre los 64 caracteres completos, haciendo imposible inferir el token correcto por medición de tiempo de respuesta:

Listing 12: PHP: Generación y validación CSRF con `hash_equals()`

```

1 <?php
2 function generateCsrfToken(): string {
3     if (empty($_SESSION['csrf_token'])) {
4         $_SESSION['csrf_token'] = bin2hex(random_bytes(32)); // 256 bits, 64 hex
5         chars
6     }
7     return $_SESSION['csrf_token'];
8 }
9 function validateCsrfToken(string $tokenRecibido): bool {

```



```

10     if (empty($_SESSION['csrf_token'])) return false;
11     return hash_equals($_SESSION['csrf_token'], $tokenRecibido); // Tiempo constante
12 }
13
14 // En el handler POST:
15 $token = filter_input(INPUT_POST, 'csrf_token', FILTER_DEFAULT) ?? '';
16 if (!validateCsrfToken($token)) {
17     http_response_code(403);
18     die('Token CSRF invalido o ausente.');
```

En Java, el equivalente se construye con `java.security.SecureRandom` (generador criptográficamente seguro certificado por FIPS 140-2) y codificación Base64 estándar, almacenando el token en `HttpSession` y validando con `MessageDigest.isEqual()` para garantizar la comparación en tiempo constante [12]:

Listing 13: Java/Servlet: Token CSRF con `SecureRandom` y Base64

```

1 // Generacion (en el Servlet GET del formulario)
2 HttpSession session = request.getSession();
3 if (session.getAttribute("csrf_token") == null) {
4     byte[] bytes = new byte[32]; // 256 bits de entropia
5     new java.security.SecureRandom().nextBytes(bytes);
6     String token = java.util.Base64.getEncoder().encodeToString(bytes);
7     session.setAttribute("csrf_token", token);
8 }
9 request.setAttribute("csrfToken", session.getAttribute("csrf_token"));
10
11 // Validacion (en el Servlet POST) -- tiempo constante con MessageDigest.isEqual
12 String tokenFormulario = request.getParameter("csrf_token");
13 String tokenSesion = (String) session.getAttribute("csrf_token");
14 if (tokenSesion == null || tokenFormulario == null ||
15     !MessageDigest.isEqual(tokenSesion.getBytes("UTF-8"),
16                             tokenFormulario.getBytes("UTF-8"))) {
17     response.sendError(HttpServletResponse.SC_FORBIDDEN, "Token CSRF invalido");
18     return;
19 }
```

2.4.3. Cabeceras de seguridad HTTP y defensa en profundidad

Las **cabeceras de respuesta HTTP** constituyen una capa adicional de defensa que opera a nivel de navegador, independiente de la lógica de la aplicación. Este enfoque de **defensa en profundidad** (*defense-in-depth*) garantiza que, incluso si una vulnerabilidad individual escapa al primer control, capas adicionales limiten su impacto [19]:

- **X-Frame-Options: DENY**: prohíbe que la página sea incrustada en un `<iframe>` de cualquier origen, neutralizando ataques de *clickjacking*.
- **X-Content-Type-Options: nosniff**: impide que el navegador “adivine” el tipo MIME de un recurso ignorando el `Content-Type` declarado, lo que podría permitir ejecutar un archivo de texto como JavaScript.
- **Content-Security-Policy (CSP)**: define una lista blanca de orígenes permitidos para scripts, estilos, imágenes y otros recursos. `default-src 'self'` bloquea la carga de cualquier recurso externo no declarado. `frame-ancestors 'none'` es el sucesor moderno de `X-Frame-Options`. Una CSP bien configurada reduce drás-

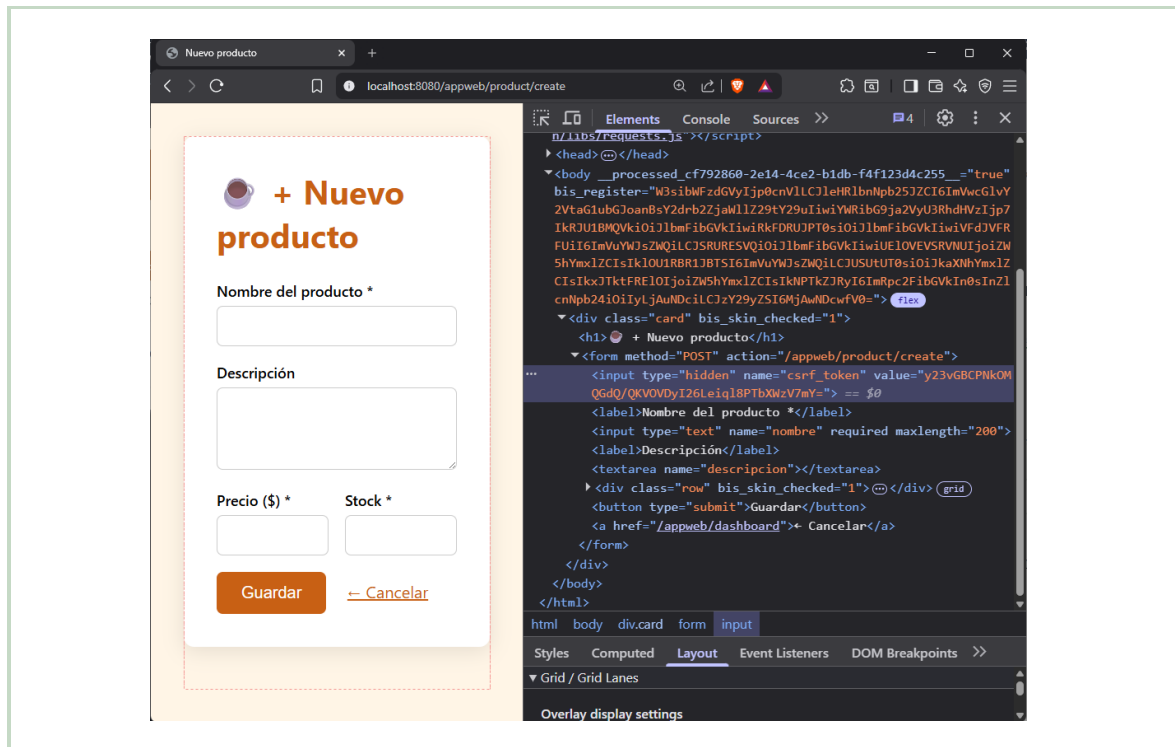


Figura 2: Inspección del formulario en DevTools del navegador. El campo oculto `csrf_token` contiene 64 caracteres hexadecimales generados criptográficamente en PHP, y cadenas Base64 de 44 caracteres en Java. Ambos son distintos en cada sesión e imposibles de predecir.

ticamente el impacto de vulnerabilidades XSS residuales.

- **Referrer-Policy: strict-origin-when-cross-origin:** controla qué información del URL de origen se incluye en el encabezado `Referer` al navegar a sitios externos, protegiendo la privacidad de los usuarios y evitando la filtración de URLs internas con tokens en la cadena de consulta.

Listing 14: PHP: Cabeceras de seguridad HTTP en `includes/headers.php`

```

1 <?php
2 function sendSecurityHeaders(): void {
3     header('X-Content-Type-Options: nosniff');
4     header('X-Frame-Options: DENY');
5     header('X-XSS-Protection: 1; mode=block');
6     header("Content-Security-Policy: default-src 'self'; " .
7         "script-src 'self'; style-src 'self' 'unsafe-inline'; " .
8         "img-src 'self' data:; frame-ancestors 'none';");
9     header('Referrer-Policy: strict-origin-when-cross-origin');
10    header_remove('X-Powered-By'); // Oculta la version de PHP al atacante
11    // En produccion HTTPS:
12    // header('Strict-Transport-Security: max-age=31536000; includeSubDomains;
13    // preload');
14 }
```

Listing 15: Java/Servlet: Filtro global de cabeceras HTTP `@WebFilter("/*")`

```

1 @WebFilter("/*")
2 public class SecurityHeadersFilter implements Filter {
3     @Override
4     public void doFilter(ServletRequest req, ServletResponse res,
```

```

5         FilterChain chain) throws IOException, ServletException {
6             HttpServletResponse response = (HttpServletResponse) res;
7             response.setHeader("X-Content-Type-Options", "nosniff");
8             response.setHeader("X-Frame-Options", "DENY");
9             response.setHeader("X-XSS-Protection", "1; mode=block");
10            response.setHeader("Content-Security-Policy",
11                "default-src 'self'; script-src 'self'; " +
12                "style-src 'self' 'unsafe-inline'; img-src 'self' data:; " +
13                "frame-ancestors 'none';");
14            response.setHeader("Referrer-Policy", "strict-origin-when-cross-origin");
15            chain.doFilter(req, res);
16        }
17    }

```

La ventaja del enfoque Java con `@WebFilter` es que las cabeceras se aplican **automáticamente a todas las respuestas** sin necesidad de incluir una llamada explícita en cada archivo. En PHP, el desarrollador debe recordar incluir `sendSecurityHeaders()` en cada archivo de entrada, lo que introduce un riesgo humano de omisión.

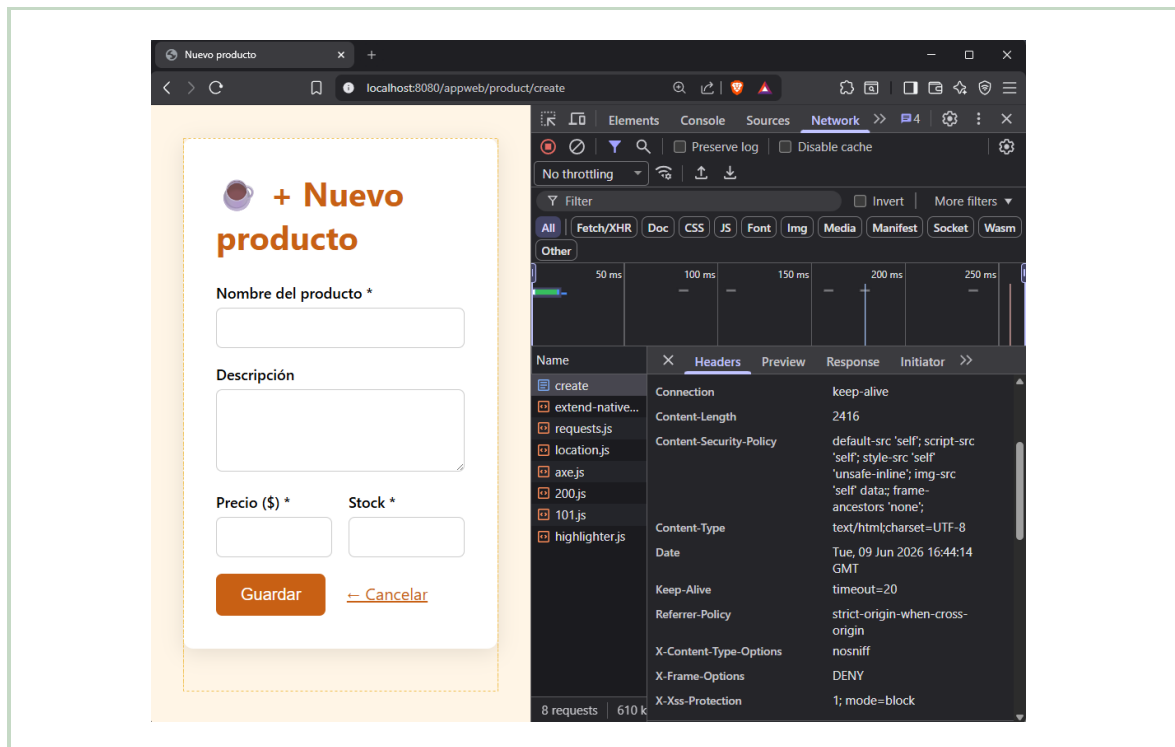


Figura 3: Panel Network de DevTools mostrando las cabeceras de respuesta HTTP. Se aprecian `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Content-Security-Policy` y la ausencia de `X-Powered-By` en la aplicación PHP.

2.5. Arquitectura de Seguridad: Spring Boot y ASP.NET Core

Las dos tecnologías alternativas más relevantes para el desarrollo backend moderno — **Spring Boot** (opción Java empresarial) y **ASP.NET Core** (plataforma Microsoft)— proponen soluciones arquitectónicas que evolucionan los conceptos implementados en esta práctica con Servlets puros hacia modelos de mayor abstracción y convención sobre

configuración. Comprender estas arquitecturas permite evaluar la progresión natural del ecosistema Java más allá de Jakarta EE.

2.5.1. Pipeline de middleware: ASP.NET Core vs Spring Security

ASP.NET Core organiza el procesamiento de peticiones HTTP mediante un **pipeline de middleware** que implementa el patrón *Chain of Responsibility*: cada componente del pipeline puede procesar la petición, pasarla al siguiente componente o cortocircuitar la cadena. El orden de registro es crítico para la seguridad y el comportamiento de la aplicación:

Listing 16: ASP.NET Core: orden recomendado del pipeline de middleware

```

1 // Program.cs -- el orden de los Use* determina la cadena de procesamiento
2 var app = builder.Build();
3
4 app.UseHttpsRedirection(); // 1. Redirigir HTTP a HTTPS (antes de todo)
5 app.UseStaticFiles();     // 2. Servir archivos estaticos sin autenticacion
6 app.UseRouting();         // 3. Resolver la ruta antes de autenticar
7 app.UseAuthentication();   // 4. Identificar QUIEN es el usuario
8 app.UseAuthorization();    // 5. Verificar QUE puede hacer (debe ir despues de
    Auth)
9 app.MapControllers();      // 6. Delegar al controlador correspondiente
10
11 // INCORRECTO: UseAuthorization() antes de UseAuthentication()
12 // -- el usuario nunca estaria identificado al verificar permisos

```

El orden importa porque `UseAuthentication()` establece el contexto de identidad del usuario (`ClaimsPrincipal`) que `UseAuthorization()` necesita para evaluar políticas. Si se invierten, la autorización opera sobre un usuario anónimo y todos los endpoints protegidos quedarían inaccesibles o, peor, accesibles sin restricción según la política por defecto.

Spring Security implementa el mismo concepto mediante una cadena de filtros configurada a través de `SecurityFilterChain`. En lugar del pipeline declarativo de ASP.NET, Spring usa una **cadena de filtros de seguridad** que intercepta cada petición HTTP antes de que llegue al controlador:

Listing 17: Spring Boot: configuración de `SecurityFilterChain`

```

1 @Configuration
2 @EnableWebSecurity
3 public class SecurityConfig {
4
5     @Bean
6     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
7         http
8             // Autorizacion declarativa por ruta
9             .authorizeHttpRequests(auth -> auth
10                 .requestMatchers("/login", "/register", "/css/**").permitAll()
11                 .anyRequest().authenticated() // Resto requiere login
12             )
13             // Formulario de login integrado
14             .formLogin(form -> form
15                 .loginPage("/login")
16                 .defaultSuccessUrl("/dashboard", true)

```

```

17         .failureUrl("/login?error=true")
18     )
19     // Gestion de sesion
20     .sessionManagement(session -> session
21         .sessionFixation().newSession()           // Equivale a
22         session.invalidate()
23         .maximumSessions(1)                         // Una sesion por usuario
24     )
25     // CSRF habilitado por defecto en Spring Security
26     .csrf(csrf -> csrf.csrfTokenRepository(
27         CookieCsrfTokenRepository.withHttpOnlyFalse()));
28     return http.build();
29 }

```

La diferencia arquitectónica fundamental con el `AuthFilter.java` implementado en esta práctica (Jakarta EE puro) es que Spring Security aplica las políticas de forma **declarativa** mediante anotaciones y configuración centralizada, mientras que el `@WebFilter` de Jakarta EE requiere lógica imperativa en cada filtro. Ambos enfoques logran el mismo resultado, pero Spring Security elimina el riesgo de omitir protecciones en rutas nuevas: al declarar `anyRequest().authenticated()`, cualquier ruta no declarada explícitamente queda automáticamente protegida.

2.5.2. Inyección de Dependencias: ciclos de vida *Transient*, *Scoped* y *Singleton*

La **Inyección de Dependencias** (DI, *Dependency Injection*) es un patrón en el que un contenedor de inversión de control (IoC) gestiona la creación e inyección de dependencias entre componentes. Tanto ASP.NET Core como Spring Boot implementan contenedores DI nativos con tres ciclos de vida que determinan cuándo se crea y destruye cada instancia:

- **Transient** (nueva instancia por cada inyección): el contenedor crea un objeto nuevo cada vez que se solicita la dependencia, incluso dentro de la misma petición HTTP. Adecuado para servicios ligeros sin estado compartido, como utilidades de formato o generadores de tokens CSRF (`bin2hex(random_bytes(32))`). En ASP.NET: `services.AddTransient<ICsrfService, CsrfService>()`.
- **Scoped** (una instancia por petición HTTP): el contenedor crea el objeto una vez por petición y lo reutiliza en toda la cadena de dependencias de esa petición. Es el ciclo de vida idóneo para **repositorios de datos** como `PdoProductoRepository` o `JdbcProductoRepository`: una sola conexión de base de datos por petición evita el overhead de abrir y cerrar conexiones múltiples y garantiza que las operaciones dentro de una transacción compartan el mismo contexto. En ASP.NET: `services.AddScoped<IProductoRepository, EfProductoRepository>()`.
- **Singleton** (una instancia para toda la aplicación): el contenedor crea el objeto una sola vez al iniciar la aplicación y lo reutiliza en todas las peticiones y todos los hilos. Apropiado para servicios que gestionan estado global inmutable o recursos costosos de inicializar: cachés en memoria, clientes HTTP (`HttpClient`), configuración de la aplicación. En ASP.NET: `services.AddSingleton<ICacheService, MemoryCacheService>()`.

Listing 18: Spring Boot: registro de dependencias con distintos ciclos de vida

```

1 @Configuration
2 public class AppConfig {
3
4     // SINGLETON: configuracion global, inicializada una vez
5     @Bean
6     public PasswordEncoder passwordEncoder() {
7         return new BCryptPasswordEncoder(12); // cost=12, misma config que jBCrypt
8     }
9
10    // SCOPED (request): una conexion de BD por peticion HTTP
11    // Spring gestiona apertura y cierre automaticamente
12    @Bean
13    @RequestScope
14    public ProductoRepository productoRepository(DataSource dataSource)
15        throws SQLException {
16        return new JdbcProductoRepository(dataSource.getConnection());
17    }
18 }
19
20 // TRANSIENT via @Component: nueva instancia cada vez que se inyecta
21 @Component
22 @Scope("prototype") // prototype = Transient en terminologia Spring
23 public class CsrfTokenGenerator {
24     public String generate() {
25         byte[] bytes = new byte[32];
26         new SecureRandom().nextBytes(bytes);
27         return Base64.getEncoder().encodeToString(bytes);
28     }
29 }

```

En el contexto de esta práctica, el repositorio `JdbcProductoRepository` debería tener ciclo de vida **Scoped**: cada petición HTTP requiere una conexión de base de datos limpia y aislada. Si fuera **Singleton**, una conexión compartida entre hilos concurrentes produciría condiciones de carrera (*race conditions*) en las transacciones. Si fuera **Transient**, se abrirían conexiones múltiples innecesarias dentro de la misma petición.

2.5.3. Spring IoC Container y Spring Security: comparativa con Jakarta EE

El **contenedor IoC de Spring** (*ApplicationContext*) gestiona el ciclo de vida completo de los componentes de la aplicación, denominados *beans*. A diferencia del enfoque imperativo de Jakarta EE puro (donde el desarrollador instancia los objetos con `new`), Spring utiliza **anotaciones declarativas** que el contenedor procesa en tiempo de inicio:

Listing 19: Comparativa: Jakarta EE manual vs Spring Boot declarativo

```

1 // == JAKARTA EE (implementacion de esta practica) -- instanciacion manual ==
2 @WebServlet("/dashboard")
3 public class DashboardServlet extends HttpServlet {
4     @Override
5     protected void doGet(HttpServletRequest req, HttpServletResponse res)
6         throws ServletException, IOException {
7         // El desarrollador gestiona manualmente la conexion y el repositorio
8         try (Connection conn = DBConnection.get()) {
9             ProductoRepositoryInterface repo = new JdbcProductoRepository(conn);
10            List<Map<String, Object>> productos = repo.getAllByUsuarioId(userId);

```

```

11         req.setAttribute("productos", productos);
12     }
13     req.getRequestDispatcher("/views/dashboard.jsp").forward(req, res);
14 }
15 }
16
17 // === SPRING BOOT -- inyeccion declarativa via @Autowired / constructor ===
18 @Controller
19 public class DashboardController {
20
21     private final ProductoRepository productoRepository; // Inyectado por Spring
22
23     // Inyeccion por constructor (practica recomendada en Spring 5+)
24     public DashboardController(ProductoRepository productoRepository) {
25         this.productoRepository = productoRepository;
26     }
27
28     @GetMapping("/dashboard")
29     // @PreAuthorize("isAuthenticated()") -- Spring Security evalua automaticamente
30     public String dashboard(Model model, Principal principal) {
31         int userId = ((CustomUser) ((Authentication)
32 principal).getPrincipal()).getId();
33         model.addAttribute("productos", productoRepository.findByUsuarioId(userId));
34         return "dashboard"; // Resuelve a templates/dashboard.html (Thymeleaf)
35     }
36 }

```

Spring Security extiende el contenedor IoC con una arquitectura de autenticación en capas. Sus tres componentes principales son:

- **UserDetailsService**: interfaz que Spring Security llama para cargar los datos del usuario desde cualquier fuente (base de datos, LDAP, servicio externo). Equivale a la consulta SQL que realiza `LoginServlet.java` en esta práctica: `SELECT * FROM usuarios WHERE email = ?`. La implementación concreta devuelve un objeto `UserDetails` con el hash de contraseña que Spring verifica automáticamente con `BCryptPasswordEncoder`.
- **AuthenticationManager**: orquesta el proceso de autenticación delegando en uno o más `AuthenticationProvider`. El proveedor por defecto (`DaoAuthenticationProvider`) combina `UserDetailsService` con `PasswordEncoder` para comparar la contraseña ingresada con el hash almacenado en tiempo constante, equivalente al `BCrypt.checkpw()` implementado en `LoginServlet.java`.
- **SecurityFilterChain**: la cadena de filtros descrita en el apartado anterior. Reemplaza el `AuthFilter.java` manual que protege todas las rutas con `@WebFilter("/**")` en la implementación de esta práctica.

La ventaja de Spring Security sobre el enfoque de Servlets puros es que elimina **toda la lógica de autenticación boilerplate**: el desarrollador declara *qué* proteger (rutas y políticas) sin necesidad de implementar *cómo* (gestión de sesiones, comparación de hashes, CSRF). En esta práctica se implementó equivalentes manuales de todos estos mecanismos para demostrar comprensión de los principios subyacentes.

2.5.4. Principios SOLID aplicados al backend del PFC

Los principios **SOLID** constituyen las cinco guías fundamentales del diseño orientado a objetos que mejoran la mantenibilidad, testabilidad y extensibilidad del código. Se analizan a continuación con ejemplos concretos de los archivos reales del repositorio de esta práctica:

S — Principio de Responsabilidad Única (*Single Responsibility Principle*, SRP): cada clase debe tener una única razón para cambiar. En el backend Java del PFC, `AuthFilter.java` tiene exclusivamente la responsabilidad de verificar la existencia de `user_id` en la sesión y redirigir si no existe —no consulta la base de datos, no genera tokens, no renderiza vistas—. Asimismo, `SecurityHeadersFilter.java` solo inyecta cabeceras HTTP y `JdbcProductoRepository.java` solo traduce operaciones de negocio a SQL. Si alguna de estas clases debiera cambiar por una razón diferente a su responsabilidad declarada, sería una violación de SRP.

O — Principio Abierto/Cerrado (*Open/Closed Principle*, OCP): las entidades de software deben estar abiertas a la extensión pero cerradas a la modificación. En el PFC, `DashboardServlet.java` depende de `ProductoRepositoryInterface`, no de `JdbcProductoRepository`. Esto significa que se puede agregar una implementación alternativa —por ejemplo, `CacheProductoRepository` que consulte Redis antes de ir a MySQL— sin modificar una sola línea del Servlet. El Servlet está *cerrado* a la modificación pero *abierto* a recibir nuevas implementaciones del repositorio.

L — Principio de Sustitución de Liskov (*Liskov Substitution Principle*, LSP): las implementaciones concretas deben ser sustituibles por su interfaz sin alterar la corrección del programa. Cualquier implementación de `ProductoRepositoryInterface` —ya sea `JdbcProductoRepository` en producción o una hipotética `InMemoryProductoRepository` en pruebas unitarias— debe comportarse de manera contractualmente equivalente: `getAllByUsuarioId()` siempre devuelve una lista (jamás nulo), `delete()` siempre devuelve `true` si y solo si se eliminó exactamente una fila. Una implementación que lanzara una excepción no declarada o devolviera `null` violaría LSP.

I — Principio de Segregación de Interfaces (*Interface Segregation Principle*, ISP): los clientes no deben depender de métodos que no utilizan. La interfaz `ProductoRepositoryInterface` expone exclusivamente los cinco métodos CRUD necesarios (`getAllByUsuarioId`, `getByIdAndUsuarioId`, `create`, `update`, `delete`). No contiene métodos de exportación a CSV, generación de reportes o notificaciones por correo que `DashboardServlet` nunca necesitaría. Una interfaz “gorda” que incluyera esos métodos obligaría a `JdbcProductoRepository` a implementarlos aunque fueran irrelevantes, violando ISP.

D — Principio de Inversión de Dependencias (*Dependency Inversion Principle*, DIP): los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones. En el PFC, `DashboardServlet.java` (módulo de alto nivel: lógica de negocio) depende de `ProductoRepositoryInterface` (abstracción), no de `JdbcProductoRepository` (módulo de bajo nivel: SQL y JDBC). Esta inversión permite sustituir la implementación de persistencia —cambiar MySQL por PostgreSQL, o JDBC por un ORM— sin tocar el Servlet. La inyección se realiza de forma explícita en cada método del Servlet:

Listing 20: DIP en acción: `DashboardServlet` depende de la abstracción, no de la implementación concreta


```
1 // CORRECTO (DIP aplicado): DashboardServlet depende de la INTERFAZ
2 ProductoRepositoryInterface repo = new JdbcProductoRepository(conn);
3 //
4 //          abstraccion          implementacion concreta
5 // Si mañana cambiamos a PostgresProductoRepository, DashboardServlet NO cambia.
6
7 // VIOLACION DE DIP (lo que NO debe hacerse):
8 JdbcProductoRepository repo = new JdbcProductoRepository(conn);
9 // Acopla el Servlet directamente a la implementacion concreta JDBC.
10 // Cambiar la BD o el ORM implica modificar todos los Servlets.
```

Resumen SOLID en el PFC: SRP — cada clase tiene una sola razón para cambiar (AuthFilter solo autentica, Repository solo persiste). OCP — DashboardServlet acepta nuevas implementaciones del repositorio sin modificarse. LSP — cualquier implementación de `ProductoRepositoryInterface` es sustituible en producción o pruebas. ISP — la interfaz expone solo los 5 métodos CRUD necesarios. DIP — los Servlets dependen de la interfaz abstracta, no de JDBC directamente.

3. Arquitectura del Sistema

3.1. Repositorio Centralizado y Flujo de Trabajo Git

El código fuente completo de ambos ecosistemas —PHP 8.x y Java/JSP (Jakarta EE)— está alojado en un **Monorepo** centralizado disponible públicamente en GitHub bajo la siguiente dirección:

<https://github.com/mloorm14/practica-experimental-2-web.git>

El repositorio sigue una estrategia de **trunk-based development**: todos los integrantes del equipo (Castro Espinoza, Escudero Plaza y Loor Medranda) realizan sus contribuciones directamente sobre la rama principal (**main**) o a través de ramas de corta vida (*short-lived feature branches*) que se fusionan en un plazo máximo de 24 horas. Cada *commit* sigue la convención de mensajes semánticos (*Conventional Commits*): **feat:**, **fix:**, **docs:**, **refactor:**, permitiendo trazar con exactitud la autoría de cada cambio y generar un historial auditable. La estructura del monorepo segrega ambos ecosistemas en subdirectorios raíz independientes (**unidad2_php/** y **unidad2_java/**), compartiendo el directorio **database/** con el esquema SQL común, lo que refleja la arquitectura de base de datos compartida descrita en este informe. Adicionalmente, el directorio **docs/adr/** contiene los *Architecture Decision Records* (ADR) requeridos por la guía experimental: **ADR-001-tecnologia-backend.md** y **ADR-002-estrategia-bd.md**.

3.2. Esquema de Base de Datos MySQL

Ambas aplicaciones comparten el mismo esquema de base de datos, lo que permite demostrar la interoperabilidad de las tecnologías de servidor con una misma fuente de verdad.

Listing 21: MySQL: Esquema completo de la base de datos compartida

```

1  -- Ejecutar en phpMyAdmin o en terminal MySQL:
2  -- mysql -u root -p < database/schema.sql
3
4  CREATE DATABASE IF NOT EXISTS appweb_unidad2
5     CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
6
7  USE appweb_unidad2;
8
9  -- Tabla de usuarios (autenticacion)
10 CREATE TABLE IF NOT EXISTS usuarios (
11     id            INT AUTO_INCREMENT PRIMARY KEY,
12     nombre        VARCHAR(100) NOT NULL,
13     email         VARCHAR(150) NOT NULL UNIQUE,
14     password_hash VARCHAR(255) NOT NULL,
15     created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
16 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
17
18 -- Tabla de productos (modulo CRUD)
19 CREATE TABLE IF NOT EXISTS productos (
20     id            INT AUTO_INCREMENT PRIMARY KEY,
21     nombre        VARCHAR(200) NOT NULL,
22     descripcion   TEXT,
23     precio        DECIMAL(10,2) NOT NULL DEFAULT 0.00,

```

```

24 stock          INT          NOT NULL DEFAULT 0,
25 usuario_id     INT          NOT NULL,
26 created_at     TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
27 updated_at     TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
28                  ON UPDATE CURRENT_TIMESTAMP,
29 CONSTRAINT fk_producto_usuario
30     FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
31     ON DELETE CASCADE
32 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
33
34 CREATE INDEX IF NOT EXISTS idx_productos_usuario ON productos(usuario_id);

```

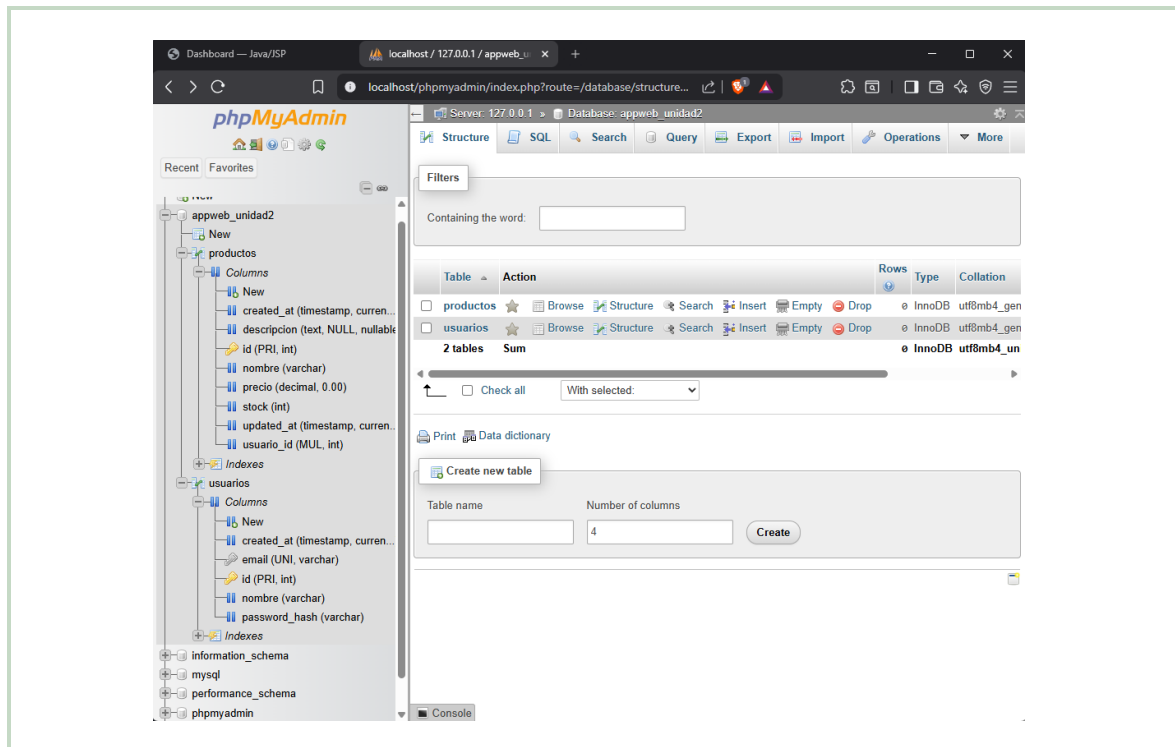


Figura 4: Vista de la base de datos *appweb_unidad2* en phpMyAdmin. Se aprecian las dos tablas (*usuarios* y *productos*), el motor InnoDB y la relación de clave foránea entre ellas.

3.3. Estructura de la Aplicación PHP

Listing 22: Estructura de directorios de la aplicación PHP

```

1 C:\xampp\htdocs\unidad2_php\
2   config\
3     db.php          <- Conexion PDO (DSN, opciones, singleton)
4   includes\
5     auth.php        <- registerUser(), loginUser(), isLoggedIn(), esc()
6     csrf.php        <- generateCsrfToken(), validateCsrfToken()
7     headers.php     <- sendSecurityHeaders()
8   repositories\
9     ProductoRepositoryInterface.php <- Contrato abstracto (interfaz)
10    PdoProductoRepository.php      <- Implementacion concreta con PDO
11  index.php                <- Redirige a login o dashboard segun sesion
12  register.php             <- Formulario + handler de registro
13  login.php                <- Formulario + handler de login

```

```

14  logout.php          <- Destruccion de sesion y redireccion
15  dashboard.php      <- Lista productos (READ)
16  create.php         <- Formulario + handler de alta (CREATE)
17  edit.php           <- Formulario + handler de edicion (UPDATE)
18  delete.php         <- Handler de eliminacion (DELETE) -- solo POST

```

3.4. Estructura de la Aplicación Java/JSP

Listing 23: Estructura del proyecto Maven para Java/JSP (Tomcat 10)

```

1  unidad2_java\
2  pom.xml          <- Dependencias: servlet-api, JSTL, MySQL, jBCrypt
3  src\main\
4    java\com\uteq\appweb\
5      config\
6        DBConnection.java      <- Conexion con DriverManager
7      filter\
8        AuthFilter.java        <- @WebFilter: redirige si no hay sesion
9        SecurityHeadersFilter.java <- @WebFilter: inyecta cabeceras HTTP
10     repository\
11       ProductoRepositoryInterface.java <- Contrato abstracto
12       JdbcProductoRepository.java    <- Implementacion JDBC
13     servlet\
14       LoginServlet.java    <- @WebServlet("/login")
15       RegisterServlet.java <- @WebServlet("/register")
16       LogoutServlet.java  <- @WebServlet("/logout")
17       DashboardServlet.java <- @WebServlet("/dashboard")
18       CreateServlet.java   <- @WebServlet("/product/create")
19       EditServlet.java     <- @WebServlet("/product/edit")
20       DeleteServlet.java   <- @WebServlet("/product/delete")
21     webapp\
22       WEB-INF\
23         web.xml            <- Descriptor de despliegue
24       views\
25         login.jsp          <- Vista JSTL + EL
26         register.jsp
27         dashboard.jsp
28         create.jsp
29         edit.jsp

```

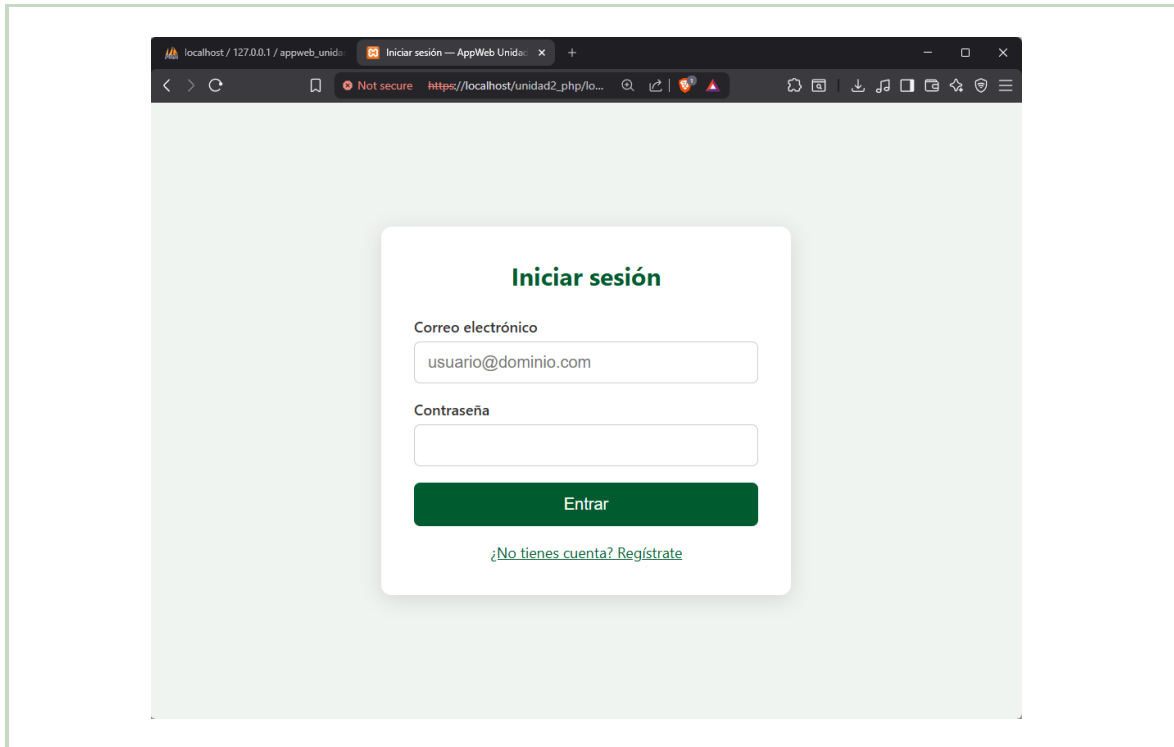


Figura 5: *Formulario de registro de la aplicación PHP con campos **nombre**, **email** y **password**, validación HTML5 y token CSRF oculto.*

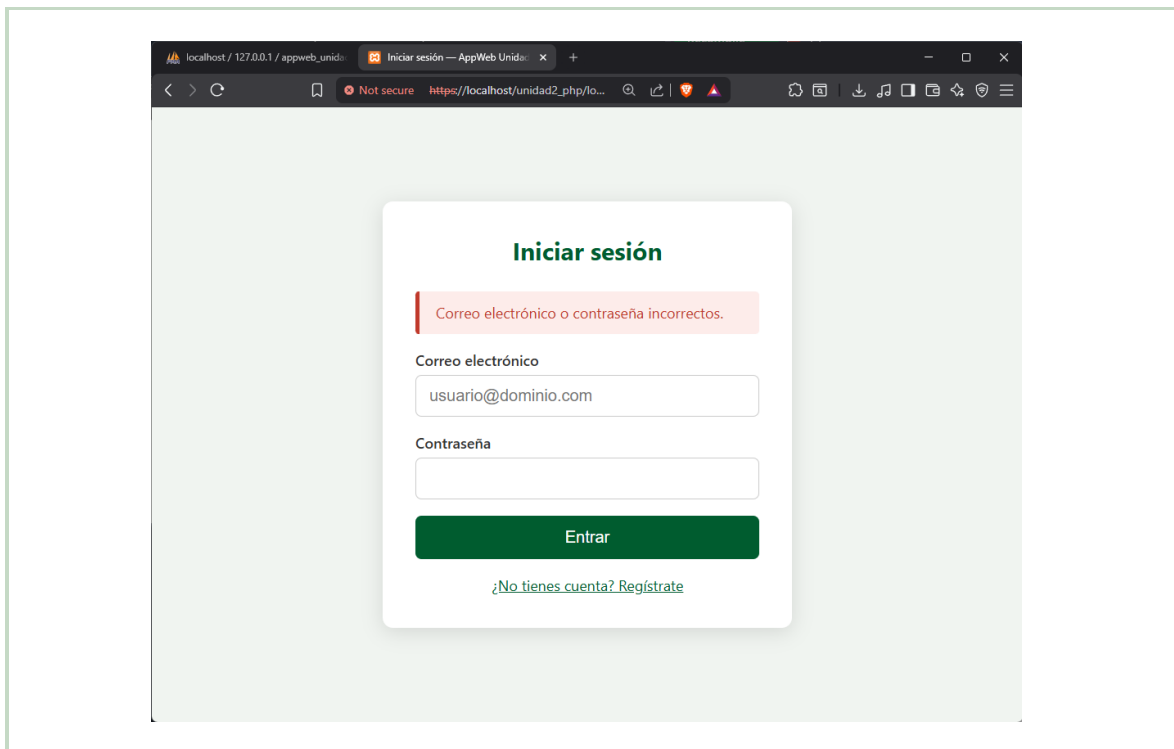


Figura 6: *Pantalla de inicio de sesión en PHP. Los mensajes de error son genéricos para no revelar qué campo falló (protección contra enumeración de usuarios).*

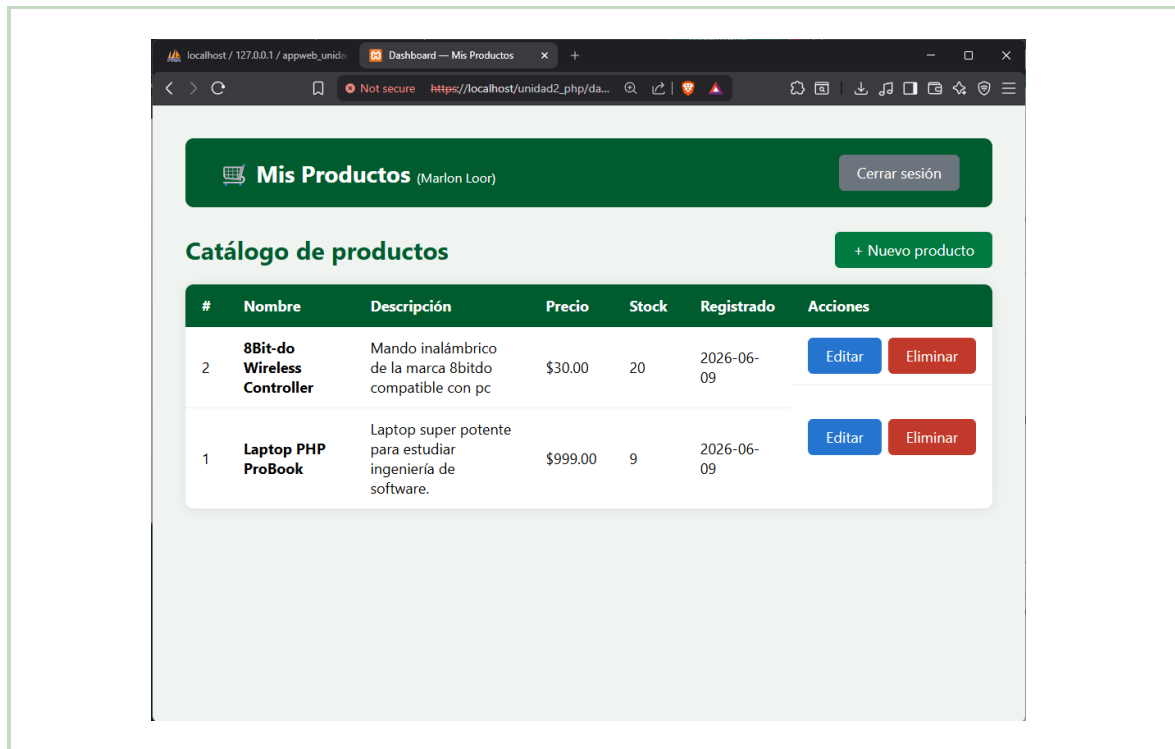


Figura 7: *Dashboard CRUD en PHP. Tabla de productos con botones Editar y Eliminar. Solo se muestran los productos del usuario autenticado (WHERE usuario_id = ?).*

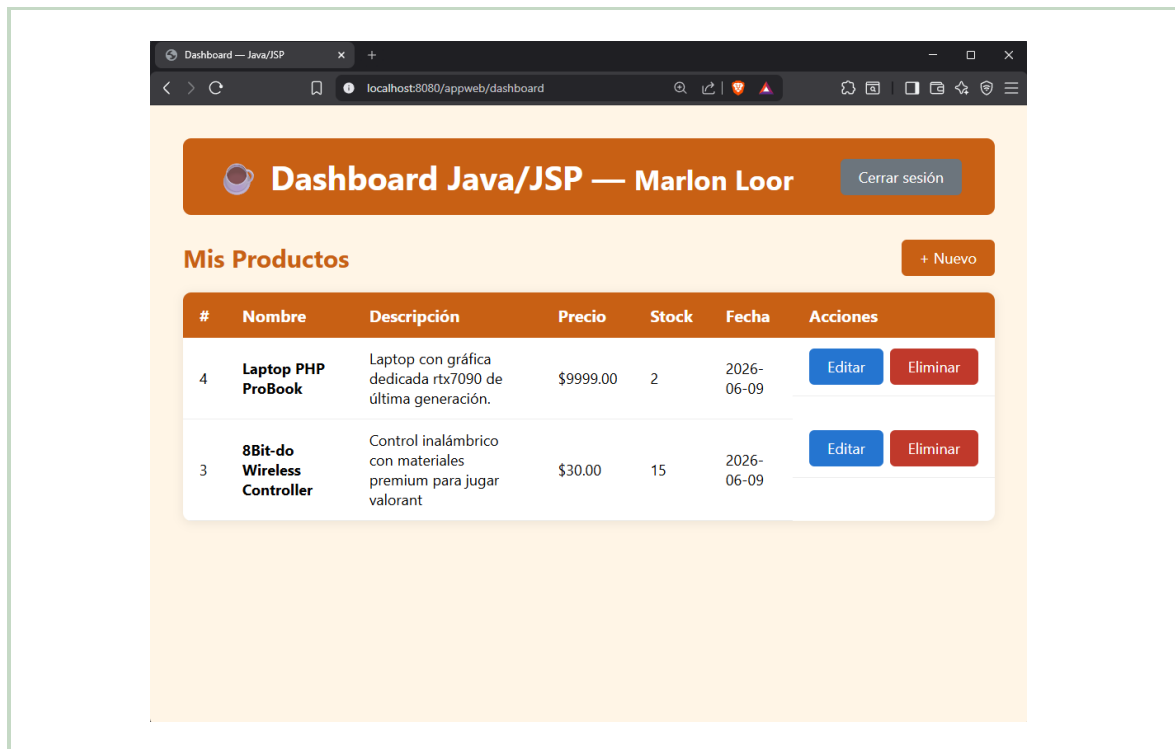


Figura 8: *Dashboard CRUD en Java/JSP renderizado por un Servlet que inyecta los datos como atributos de request (request.setAttribute(...)).*

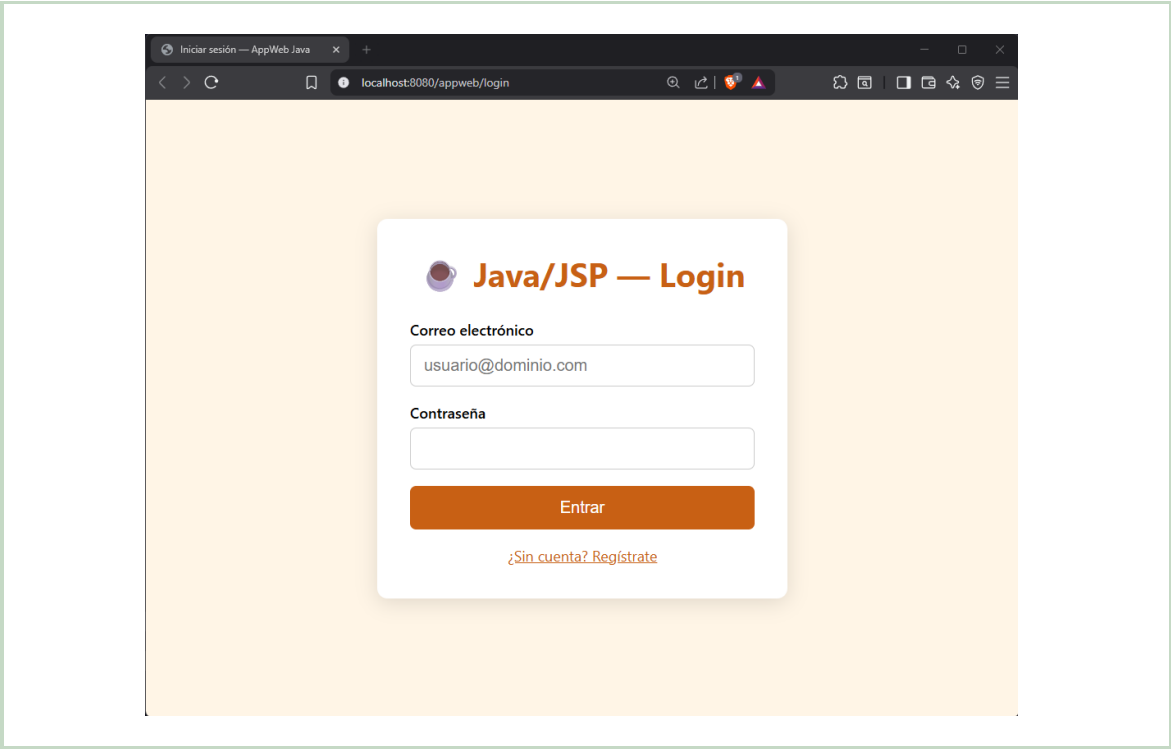


Figura 9: Formulario de login de la aplicación Java/JSP renderizado por `login.jsp`. Los datos son procesados por `LoginServlet.doPost()` con `PreparedStatement` y `BCrypt`.

4. Tabla Comparativa: PHP 8.x vs. Java/JSP (Jakarta EE)

Criterio	PHP 8.x	Java/JSP (Jakarta EE)
1. Curva de aprendizaje	Baja. Sintaxis tipo C, tipado dinámico, documentación oficial en español (php.net/manual/es). Un estudiante puede desplegar un CRUD funcional en XAMPP en pocas horas. Ecosistema accesible: WordPress, Laravel, Symfony.	Media-alta. Requiere dominio de tipado estático, anotaciones (<code>@WebServlet</code> , <code>@WebFilter</code>), ciclo de vida del Servlet, Maven, <code>web.xml</code> y la JVM. Curva empinada al inicio, pero recompensa con una arquitectura más robusta. Ecosistema: Spring Boot, Jakarta EE, Quarkus.
2. Líneas de código (Auth/CRUD)	Más compacto. Un CRUD autenticado completo cabe en ~350–500 líneas entre todos los archivos PHP. <code>password_hash()</code> y <code>filter_input()</code> reducen el código boilerplate de seguridad significativamente.	Más verboso. El mismo CRUD requiere ~800–1200 líneas: un Servlet por operación, JSP separados, filtros, clases de configuración de BD y dependencias Maven. La verbosidad mejora la legibilidad y mantenibilidad en proyectos grandes.

continúa en la página siguiente...

Criterio	PHP 8.x	Java/JSP (Jakarta EE)
3. Seguridad por defecto	Moderada. PHP no activa tipos estrictos por defecto (<code>declare(strict_types=1)</code> es opt-in). Las funciones de seguridad (<code>password_hash</code> , <code>filter_input</code>) existen pero el desarrollador debe usarlas explícitamente. Sin marco de trabajo, cada control (CSRF, cabeceras) debe codificarse manualmente.	Alta. Tipado estático obliga al compilador a detectar errores de tipo en tiempo de compilación. Los filtros <code>@WebFilter</code> aplican controles globales automáticamente. La arquitectura multi-capas (fuente → byte-code → JIT nativo) añade capas de validación que PHP no tiene de forma nativa.
4. Rendimiento (Requests/seg)	Adecuado para tráfico moderado. En benchmarks típicos con CRUD simple, PHP 8.x + PHP-FPM + Nginx alcanza ~3,000–8,000 req/s (dependiendo del hardware y la complejidad de la lógica). El JIT de PHP 8.0 mejora tareas CPU-intensivas pero no el I/O-bound web.	Superior bajo carga sostenida. Tomcat con pool de hilos maneja ~5,000–15,000 req/s en el mismo hardware tras el calentamiento de la JVM. La compilación JIT de la JVM optimiza los caminos calientes a código nativo, con mejora progresiva. El cold start es su mayor debilidad (~5–20 s).
5. Ecosistema de paquetes/librerías	Maduro y amplio. Composer gestiona ~340,000 paquetes en Packagist. Laravel/Symfony tienen ecosistemas completos. La mayoría de integraciones web tienen soporte nativo (Stripe, Mailgun, AWS SDK).	El mayor y más maduro. Maven Central alberga >500,000 artefactos. El ecosistema Spring (Boot, Security, Data, Cloud) cubre prácticamente cualquier necesidad empresarial. Integraciones con Apache Kafka, Elasticsearch y Kubernetes son de primera clase [20].
6. Hosting y desplegabilidad (Ecuador)	Ventaja clara. El 95 % de los proveedores de hosting compartido en Ecuador (GoDaddy Ecuador, Hostgator Latam, Hostinger Ecuador) incluyen PHP + MySQL por defecto desde ~\$3–6/mes. Despliegue por FTP/SFTP sin conocimientos de contenedores. Ideal para proyectos de alcance local y PYMES.	Limitado en hosting compartido. Requiere VPS o PaaS (DigitalOcean, Railway, Render) con JDK instalado, lo que eleva el costo a ~\$12–25/mes. Los proveedores ecuatorianos raramente ofrecen soporte Java/Tomcat. Viable con Docker/Railway para proyectos con presupuesto.

continúa en la página siguiente...

Criterio	PHP 8.x	Java/JSP (Jakarta EE)
7. Comunidad	Grande y accesible. PHP es el lenguaje con mayor presencia en Stack Overflow en el contexto web (más de 1.5 millones de preguntas etiquetadas). La comunidad hispanohablante es especialmente activa: documentación oficial completa en español en php.net/manual/es , foros de Laravel en español, canales de Symfony en LATAM y una alta densidad de tutoriales en YouTube en español. Packagist y GitHub concentran miles de proyectos de código abierto activos. Para Ecuador, la comunidad local de PHP es la más accesible para encontrar colaboradores y soporte técnico.	Madura y empresarial. Java es históricamente la etiqueta más consultada en Stack Overflow (más de 1.8 millones de preguntas). La Eclipse Foundation, Oracle Java Champions y la Spring Community son comunidades globales con soporte de primera clase. Jakarta EE cuenta con respaldo institucional de IBM, Red Hat y Oracle. Java LTS (Java 21, soporte hasta 2031) garantiza estabilidad de largo plazo para proyectos corporativos. La comunidad es más técnica y orientada a arquitecturas empresariales, con menos presencia local en Ecuador pero mayor proyección en banca y gobierno.
8. Facilidad de pruebas unitarias	Factible con esfuerzo. PHPUnit es el estándar, pero el código procedural típico de PHP principiante dificulta el testing unitario. El patrón Repository y la inyección de dependencias (como se implementa en esta práctica) son necesarios para alcanzar cobertura real sin base de datos.	Más integrado en la arquitectura. JUnit 5 + Mockito son el estándar. La inyección de dependencias es idiomática en Java, lo que facilita reemplazar implementaciones reales por <i>mocks</i> en pruebas. Spring Boot Test simplifica aún más las pruebas de integración.
9. Integración nativa con Cloud	Buena mediante SDKs. AWS SDK for PHP, Google Cloud PHP Client y Azure SDK están disponibles vía Composer. La integración es explícita pero funcional. Serverless PHP es posible con AWS Lambda + Bref o Google Cloud Run.	Excelente y nativa. Spring Cloud integra service discovery (Eureka), configuración centralizada, circuit breakers (Resilience4j) y trazabilidad distribuida. AWS, GCP y Azure ofrecen SDK Java de primera clase. Kubernetes/GKE y las plataformas PaaS empresariales están optimizadas para workloads Java/JVM.

continúa en la página siguiente...

Criterio	PHP 8.x	Java/JSP (Jakarta EE)
10. Salario promedio (mercado Ecuador)	\$600–\$1,200/mes para desarrolladores PHP junior–mid en Ecuador (según Computrabajo Ecuador / OCC, 2024). Alta demanda en proyectos de comercio electrónico, CMS y PY-MES. Perfil más abundante y con menor barrera de entrada.	\$900–\$2,000/mes para desarrolladores Java/Spring mid–senior en Ecuador. Demanda concentrada en banca, fintech, empresas de telecomunicaciones y proyectos gubernamentales de gran escala. Perfil escaso y mejor remunerado.

Cuadro 1: Comparativa técnica ampliada entre PHP 8.x y Java/JSP (Jakarta EE 10).

5. Conclusiones

Conclusión 1 — Principios de seguridad universales. Los controles de seguridad analizados —validar toda entrada en el servidor, escapar toda salida, usar sentencias preparadas y proteger contra CSRF— son **independientes del lenguaje de backend**. El mismo principio que se aprendió con `CGI::escapeHTML()` y `$sth->execute(?)` en PERL/DBI se aplica de forma directa en `htmlspecialchars()` / PDO en PHP y en `<c:out>` / JDBC en Java. Esto confirma que los fundamentos de seguridad son transversales a cualquier tecnología.

Conclusión 2 — PHP 8.x vs. Java/JSP: fortalezas complementarias. PHP destaca por su integración nativa con Apache, su curva de aprendizaje baja y sus funciones de seguridad integradas como `password_hash()` y `filter_input()`, que reducen el margen de error humano. Java/JSP ofrece una arquitectura más robusta con filtros globales (`@WebFilter`), tipado estático y mejor rendimiento sostenido a escala. Ninguna tecnología es superior en términos absolutos: la elección depende del contexto, el equipo y los requisitos del proyecto.

Conclusión 3 — Sentencias preparadas como defensa esencial. La implementación de sentencias preparadas con marcadores de posición (?) en PDO y JDBC elimina prácticamente por completo el riesgo de inyección SQL (OWASP A03), con una puntuación CVSS 3.1 de 10.0, siempre que se desactive la emulación en PDO (`ATTR_EMULATE_PREPARES =>false`). La seguridad debe diseñarse desde la primera línea de código.

Conclusión 4 — Impacto práctico del token CSRF y las cabeceras HTTP. La implementación del token CSRF con `bin2hex(random_bytes(32))` y su validación con `hash_equals()` (PHP) o `SecureRandom` (Java) demuestra que mecanismos de protección avanzados pueden implementarse sin librerías externas. Las cabeceras de seguridad HTTP (`X-Frame-Options`, `CSP`, `X-Content-Type-Options`) constituyen una capa de defensa en profundidad que protege al usuario incluso cuando otras líneas de defensa fallan.

6. Anexo C: Evaluación Analítica

6.1. Análisis de Seguridad por Defecto

La comparación de la **línea base de seguridad** entre PHP 8.x y Java/JSP sin configuración personalizada adicional revela diferencias arquitectónicas fundamentales derivadas del diseño original de cada plataforma.

Java/JSP ofrece una línea base de seguridad más robusta por diseño estructural. El compilador Java detecta errores de tipo, referencias nulas potenciales y accesos ilegales en tiempo de compilación, antes de que el código llegue a producción. El *bytecode* generado es verificado por la JVM antes de su ejecución. El contenedor Servlet (Tomcat) separa los *web roots* por aplicación, impidiendo que una aplicación acceda a los archivos de otra. La arquitectura multi-capa (código fuente → bytecode → JIT nativo) añade tres puntos de validación que PHP, como lenguaje interpretado, no tiene de forma nativa.

Los controles que debieron codificarse manualmente en PHP (ausentes por defecto) incluyen:

- **Activación de tipos estrictos:** `declare(strict_types=1)` debe añadirse al inicio de cada archivo. Sin esto, PHP convierte tipos automáticamente ("5" se convierte a 5 en un contexto numérico), lo que puede producir comportamientos inesperados en validaciones.
- **Clase CSRF manual:** no existe ninguna protección CSRF integrada en PHP puro. La clase con `bin2hex(random_bytes(32))` y `hash_equals()` fue codificada desde cero en `includes/csrf.php`.
- **Función de escape XSS:** la función auxiliar `esc()` con `htmlspecialchars(..., ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')` debe definirse y recordarse llamar en cada punto de salida.
- **Cabeceras HTTP de seguridad:** la función `sendSecurityHeaders()` debe incluirse manualmente en cada archivo de entrada. Un archivo olvidado queda desprotegido.
- **Configuración explícita de PDO:** `PDO::ATTR_EMULATE_PREPARES =>false` debe especificarse explícitamente; de lo contrario, PDO simula las sentencias preparadas y la protección contra SQLi puede ser parcial según el driver.

En Java, los filtros `@WebFilter(/*)` aplican cabeceras de seguridad y validación de sesión a **todas** las rutas de forma declarativa; y el compilador hace imposible pasar un `String` donde se espera un `int` sin conversión explícita. La conclusión es que PHP requiere una disciplina de desarrollo más rigurosa para alcanzar el mismo nivel de seguridad base que Java ofrece por su arquitectura.

6.2. Evaluación de Factibilidad de Mercado en Ecuador

Para un **Proyecto de Fin de Carrera (PFC)** desarrollado en el contexto sociotecnológico de la provincia de Los Ríos y Ecuador en 2026, la **recomendación arquitectónica** es **PHP 8.x como plataforma principal de desarrollo**, con las siguientes justificaciones documentadas:

Infraestructura de hosting disponible y costo: la gran mayoría de proveedores de

hosting con presencia en Ecuador —incluyendo Hostinger Ecuador, GoDaddy Ecuador, Hosting Ecuador (hostingecuador.com) y Dataico— ofrecen planes de hosting compartido con PHP 8.x + MySQL desde \$3 hasta \$8 mensuales. Esta infraestructura es directamente accesible sin conocimientos de administración de servidores, contenedores Docker o gestión de la JVM. Para un proyecto universitario que debe demostrar operabilidad real, esta accesibilidad es crítica.

Disponibilidad de desarrolladores en el mercado local: según las ofertas laborales activas en Computrabajo Ecuador y LinkedIn Ecuador (junio 2026), PHP aparece en aproximadamente el 60 % de las ofertas de desarrollo web en la Costa ecuatoriana, versus un 15–20 % para Java. La mayor disponibilidad de desarrolladores PHP implica mayor facilidad para encontrar soporte, colaboradores o referentes para el proyecto.

Restricciones de conectividad y latencia: el uso de plataformas PaaS como Railway o Render (necesarias para desplegar Java/Tomcat accesiblemente) introduce latencias adicionales y dependencia de conectividad internacional estable. Un servidor PHP en hosting compartido nacional puede ofrecer latencias menores para usuarios ecuatorianos.

Recomendación con matiz: si el PFC implica un sistema de alto tráfico, arquitectura de microservicios o requiere integración con sistemas bancarios o gubernamentales, Java/Spring Boot sobre un VPS dedicado o sobre Google Cloud Run es la elección correcta, con un costo operativo de \$15–30/mes justificado por los requisitos. Para proyectos de alcance PYME, municipal o educativo en Los Ríos, PHP 8.x con Laravel en hosting compartido nacional representa la mejor relación entre costo, disponibilidad y mantenibilidad a largo plazo.

6.3. Síntesis de Análisis de Código Estático con PHPStan

PHPStan (PHP Static Analysis Tool) es una herramienta de análisis estático para PHP que detecta errores en tiempo de compilación sin ejecutar el código [21]. Opera en niveles de strictness del 0 al 9; el **nivel 5** activa verificación de tipos en parámetros, valores de retorno, acceso a propiedades y resolución de métodos en tipos desconocidos. En esta práctica se ejecutó el análisis `vendor/bin/phpstan analyse src -level=5`.

Categorías de advertencias encontradas durante el desarrollo:

1. **Incompatibilidades de tipos en parámetros:** `Parameter #1 $id of method delete() expects int, string given.` Origen: `filter_input(INPUT_POST, 'id', FILTER_VALIDATE_INT)` devuelve `int|false|null`, no `int` directamente. Corrección: `$id = (int)filter_input(...) ? : throw new InvalidArgumentException(...)`.
2. **Acceso a array sin garantía de clave:** `Offset 'password_hash' might not exist in array{id: int, ...}.` Origen: `$fila = $stmt->fetch()` devuelve `array|false`, sin que PHPStan pueda inferir las claves disponibles. Corrección: tipo anotado con `/** @var array{id: int, nombre: string, password_hash: string} $fila */` o verificación explícita con `array_key_exists()`.
3. **Resolución de métodos en tipos mixtos:** `Cannot call method fetchAll() on mixed.` Origen: el método `prepare()` de PDO devuelve `PDOStatement|false`, y sin verificar el retorno, PHPStan no puede garantizar que `fetchAll()` sea válido. Corrección: `$stmt = $pdo->prepare(...); if ($stmt === false) throw ...;`

4. **Nullability en parámetros de filtro:** Origen: `filter_input()` puede devolver null si la clave no existe en `$_POST`. Corrección: uso del operador null-coalescing: `$email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL) ?? ''`;

Refactorizaciones aplicadas para lograr reporte cero errores:

Listing 24: PHP: Anotaciones de tipo y guardas de nulabilidad para PHPStan nivel 5

```

1 <?php
2 declare(strict_types=1);
3
4 final class PDOProductoRepository implements ProductoRepositoryInterface {
5     public function __construct(private readonly PDO $pdo) {}
6
7     /** @return list<array{id:int,nombre:string,precio:string,stock:int}> */
8     public function getAllByUsuarioId(int $usuarioId): array {
9         $stmt = $this->pdo->prepare(
10             'SELECT id, nombre, precio, stock FROM productos
11              WHERE usuario_id = ? ORDER BY id DESC'
12         );
13         // Guardia explícita: prepare() puede devolver false
14         if ($stmt === false) {
15             throw new \RuntimeException('Error al preparar la consulta SQL.');

```

El resultado tras aplicar estas correcciones fue un reporte limpio de PHPStan nivel 5 con **0 errores**, confirmando que el código cumple las garantías de tipado estático requeridas. La incorporación de PHPStan en el flujo de CI/CD garantiza que futuras regresiones sean detectadas antes de fusionarse a la rama principal.

6.4. Integración de Arquitectura de Sistemas con PE-U1

La Práctica Experimental de la Unidad I (PE-U1) estableció la **capa de presentación estática** de la aplicación: los archivos HTML, CSS (con Tailwind CSS o CSS propio) y la estructura semántica de las páginas. La presente práctica (PE-U2) construye el **backend dinámico** que dota de vida a esos componentes estáticos.

La integración arquitectónica sigue el patrón **servidor-plantilla-datos** en dos modalidades:

En PHP: los archivos `.php` de lógica reemplazan a los `.html` estáticos de PE-U1 como punto de entrada. La estructura HTML de PE-U1 se incorpora dentro de las vistas PHP, envolviendo los bloques dinámicos con etiquetas PHP (`<?php foreach($productos as $p): ?>...<td><?= esc($p['nombre']) ?></td>`). Las hojas de estilo CSS y los archivos JavaScript de PE-U1 se reutilizan **sin modificación**: el contrato estático de PE-U1 se mantiene intacto.

En Java/JSP: los Servlets cargan los datos desde la base de datos y los inyectan como atributos del *request* (`request.setAttribute("productos", lista)`). Los archivos JSP de PE-U2 incorporan el HTML de PE-U1 como estructura base y sustituyen los valores estáticos por expresiones EL (`${producto.nombre}`) y etiquetas JSTL (`<c:forEach>`).

El **contrato de datos** entre el backend (PE-U2) y el frontend (PE-U1) se define como un conjunto de atributos tipados que el Servlet o el script PHP garantizan al motor de plantillas:

Contrato de datos definido entre PE-U1 y PE-U2:

- **productos:** lista con campos `id` (int), `nombre` (string), `descripcion` (string|null), `precio` (decimal/string), `stock` (int).
- **usuario_nombre:** nombre del usuario autenticado en sesión.
- **mensaje_exito/mensaje_error:** retroalimentación tras operaciones CRUD.
- **csrf_token:** token de 64 hex (PHP) o Base64 de 44 caracteres (Java) para incrustar en formularios de mutación (POST).
- **producto_editar:** objeto/array del producto seleccionado, disponible exclusivamente en la vista de edición.

Este contrato garantiza que la capa de presentación (PE-U1) es completamente intercambiable entre ambos backends sin modificar el HTML, adaptando solo las expresiones de interpolación (`<?= esc($v) ?>` en PHP vs `${v}` en JSP).

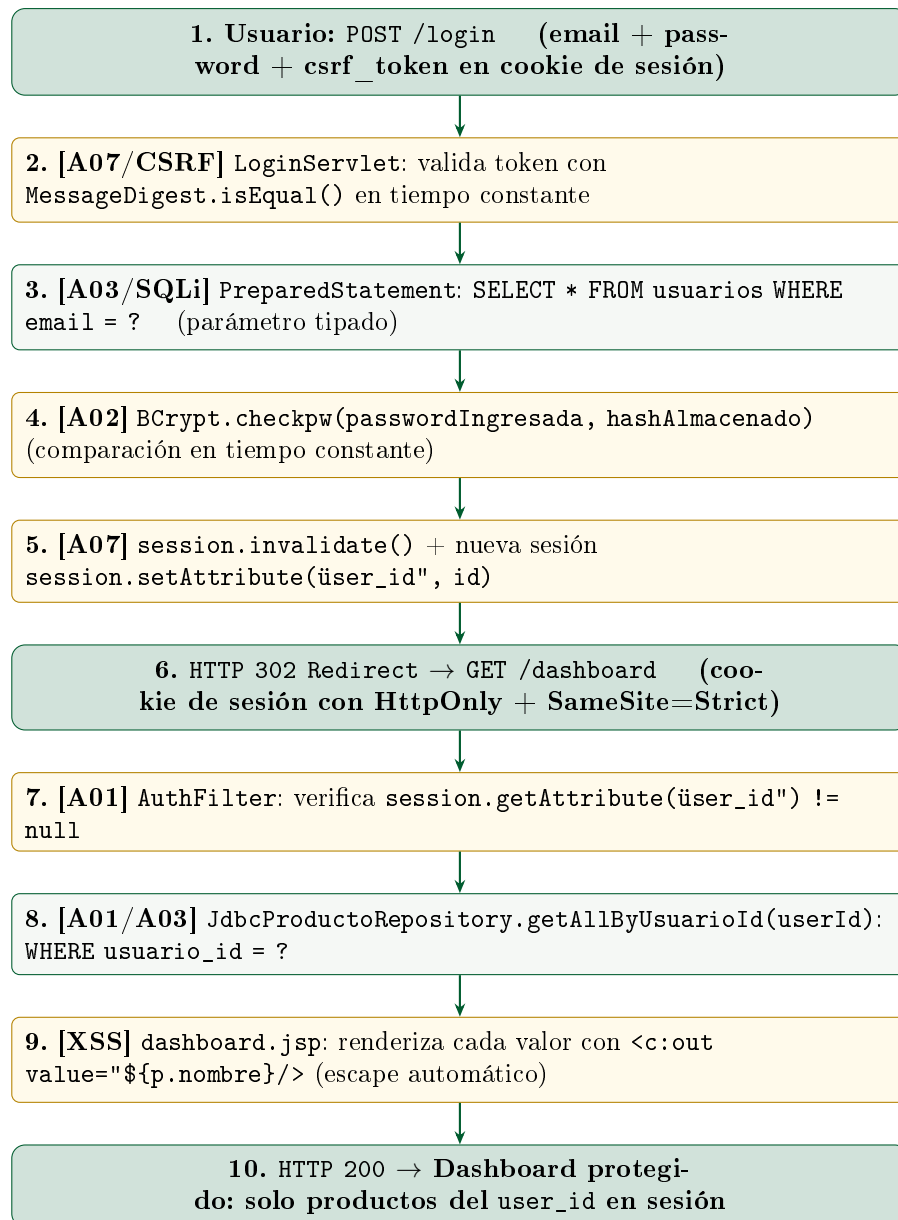


Figura 10: Diagrama de flujo de la primera petición autenticada del PFC (Java/JSP): desde el login hasta la visualización de datos protegidos. **Nodos en amarillo:** controles de seguridad OWASP (A01, A02, A07, XSS, CSRF). **Nodos en gris:** acceso a datos con sentencias preparadas (anti-SQLi A03). **Nodos en verde:** puntos de entrada/salida HTTP.

Esta integración demuestra el valor del principio de **separación de responsabilidades** aplicado a escala de proyecto: la capa de presentación (PE-U1) es responsable de la apariencia y la experiencia de usuario; la capa de acceso a datos (PE-U2) es responsable de la persistencia segura; y la capa de lógica de negocio (Servlets / scripts PHP) orquesta la comunicación entre ambas mediante contratos bien definidos.

Referencias

- [1] W3Techs. “Usage Statistics of Server-Side Programming Languages for Websites,” visitado jun. de 2026. dirección: https://w3techs.com/technologies/overview/programming_language
- [2] OWASP Foundation. “OWASP Top Ten 2021: The Ten Most Critical Web Application Security Risks,” visitado jun. de 2026. dirección: <https://owasp.org/Top10/>
- [3] D. Robinson y K. Coar, “The Common Gateway Interface (CGI) Version 1.1,” The Internet Society, RFC 3875, oct. de 2004. dirección: <https://datatracker.ietf.org/doc/html/rfc3875>
- [4] The PHP Group. “FastCGI Process Manager (FPM) — Manual de PHP,” visitado jun. de 2026. dirección: <https://www.php.net/manual/es/install.fpm.php>
- [5] Eclipse Foundation. “Jakarta EE 10 Specifications — Jakarta Servlet 6.0,” visitado jun. de 2026. dirección: <https://jakarta.ee/specifications/servlet/6.0/>
- [6] M. Hall, L. Brown e Y. Chaikin, *Core Servlets and JavaServer Pages, Vol. 2: Advanced Technologies*. Upper Saddle River, NJ, USA: Prentice Hall, 2004, ISBN: 978-0-13-140239-3.
- [7] The PHP Group. “PHP 8.0 — JIT Compiler,” visitado jun. de 2026. dirección: <https://www.php.net/releases/8.0/es.php>
- [8] R. Nixon, *Learning PHP, MySQL & JavaScript*, 6.^a ed. Sebastopol, CA, USA: O’Reilly Media, 2021, ISBN: 978-1-492-06229-9.
- [9] National Institute of Standards and Technology, “Digital Identity Guidelines: Authentication and Lifecycle Management,” NIST, Special Publication 800-63B, 2020. dirección: <https://pages.nist.gov/800-63-3/sp800-63b.html>
- [10] A. Biryukov, D. Dinu y D. Khovratovich, “Argon2: Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” Internet Engineering Task Force, RFC 9106, sep. de 2021. dirección: <https://datatracker.ietf.org/doc/html/rfc9106>
- [11] The PHP Group. “password_hash — Manual de PHP,” visitado jun. de 2026. dirección: <https://www.php.net/manual/es/function.password-hash.php>
- [12] OWASP Foundation. “Cross-Site Request Forgery Prevention Cheat Sheet,” visitado jun. de 2026. dirección: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- [13] FIRST.Org, Inc., “Common Vulnerability Scoring System v3.1: Specification Document,” FIRST, inf. téc., 2019. dirección: <https://www.first.org/cvss/v3.1/specification-document>
- [14] The PHP Group. “PDO — PHP Data Objects,” visitado jun. de 2026. dirección: <https://www.php.net/manual/es/pdo.prepared-statements.php>
- [15] L. Welling y L. Thomson, *PHP and MySQL Web Development*, 5.^a ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2016, ISBN: 978-0-321-83389-1.
- [16] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Upper Saddle River, NJ, USA: Prentice Hall, 2017, ISBN: 978-0-13-468599-1.
- [17] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008, ISBN: 978-0-13-235088-4.

- [18] OWASP Foundation. “Cross Site Scripting Prevention Cheat Sheet,” visitado jun. de 2026. dirección: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- [19] Mozilla Foundation. “HTTP Headers — Security,” visitado jun. de 2026. dirección: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers#security>
- [20] C. Walls, *Spring Boot in Action*, 2.^a ed. Shelter Island, NY, USA: Manning Publications, 2022, ISBN: 978-1-61729-454-7.
- [21] O. Mirtes. “PHPStan — PHP Static Analysis Tool,” visitado jun. de 2026. dirección: <https://phpstan.org/>

Referencias en Formato APA

Las siguientes referencias corresponden a las fuentes bibliográficas consultadas durante el desarrollo de la práctica, presentadas en formato APA 7 según lo requerido por la guía experimental.

Hall, M., Brown, L., & Chaikin, Y. (2004). *Core Servlets and JavaServer Pages, Vol. 2: Advanced Technologies*. Prentice Hall.

Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

National Institute of Standards and Technology. (2020). *Digital Identity Guidelines: Authentication and Lifecycle Management* (Special Publication 800-63B). NIST. <https://pages.nist.gov/800-63-3/sp800-63b.html>

Nixon, R. (2021). *Learning PHP, MySQL & JavaScript* (6.a ed.). O'Reilly Media.

OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>

OWASP Foundation. (2024). *Cross-Site Request Forgery Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Walls, C. (2022). *Spring Boot in Action* (2.a ed.). Manning Publications.

Welling, L., & Thomson, L. (2016). *PHP and MySQL Web Development* (5.a ed.). Addison-Wesley.